

Recurrent Neural Networks (RNNs)

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

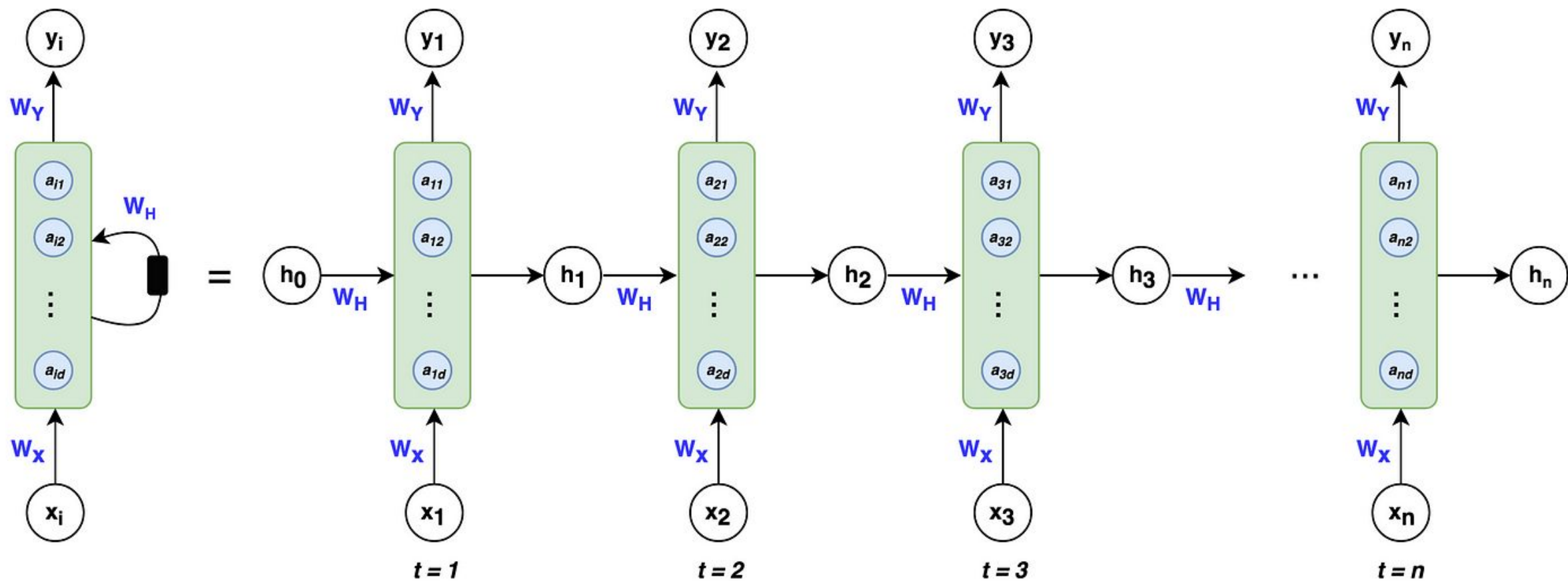


جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



LMH
Lady Margaret Hall

August 4, 2025



1. Motivation
2. Learning Outcomes
3. Introduction
4. RNN Architectures
5. RNN Training Challenges
6. Vanishing and Exploding Gradients
7. LSTM – Long Short-Term Memory
8. GRU – Gated Recurrent Unit
9. LSTM vs GRU
10. References

Why Do We Need RNNs?

- ▶ Traditional feedforward networks don't handle **sequential data** effectively.
- ▶ Many applications (**language modeling, time-series prediction, speech recognition**) require memory of past inputs.
- ▶ RNNs enable **temporal dynamic behavior** by maintaining hidden states.

Examples Where Order Matters:

- ▶ Translating “I am happy” vs “Happy I am”
- ▶ Predicting next stock price based on past trends
- ▶ Understanding a sentence word-by-word

Key Idea

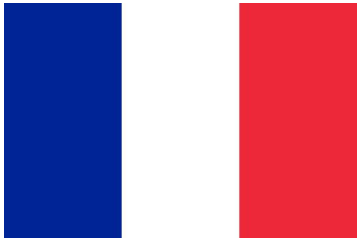
Add a feedback loop to remember previous computations — introducing memory into neural networks.

By the end of this session, you should be able to:

- ▶ Understand the structure and working of Recurrent Neural Networks (RNNs)
- ▶ Recognize various RNN architectures (One-to-Many, Many-to-One, Many-to-Many)
- ▶ Explain the concept of shared parameters in RNNs
- ▶ Explain why RNNs suffer from vanishing and exploding gradients
- ▶ Understand how LSTMs and GRUs solve these problems
- ▶ Compare LSTMs and GRUs in terms of performance and complexity
- ▶ Recognize limitations and future directions in sequence modeling

RNN: Introduction

Traditional Language Models

	Sequence	$P(\text{Sequence})$
 J'ai vu le match de foot 	I saw the game of soccer	4.5 e-5
	I saw the soccer game	<u>6.0 e-5</u>
	I saw the soccer match	4.6 e-5
	Saw I the game of soccer	2.6 e-9

$$P(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}$$

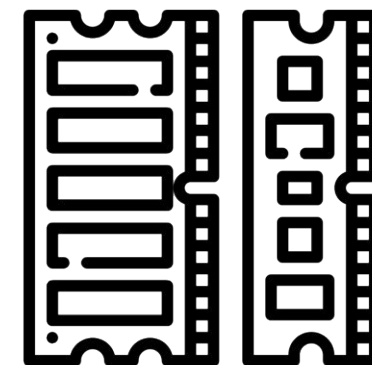
$$P(w_3|w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$$

$$P(w_1, w_2, w_3) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_2)$$

- Large N-grams to capture dependencies between distant words
- Need a lot of space and RAM

N-grams - Limitations

- N-grams **consume a lot of memory**
- **Fixed Context Window**: Only consider the previous $(n-1)$ words.
- **Data Sparsity**: Large $n \rightarrow$ exponential n -grams $\rightarrow$ many rare or unseen combinations in the training data.
- **No Learning or Adaptability**: Don't "learn" in the neural sense; they just count and normalize.
- Different types of **RNNs** are the **preferred alternative**



What is an RNN?

A neural network with loops — allowing information to persist.

Core Elements:

- ▶ **Hidden State** h_t : captures memory of previous inputs
- ▶ **Input** x_t , **Output** y_t
- ▶ Same weights used across time steps (parameter sharing)

Mathematical Formulation:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

This recurrence allows information to propagate through time.

- RNNs (Recurrent Neural Networks) are designed to handle **sequential data**—data where order matters, like language, time series, or speech.

What Problem Do RNNs Solve?

- **Capturing Sequence and Context**

- Keep a **hidden state** that carries information from previous steps, allowing them to remember **past inputs**.

Example: In a sentence, the meaning of a word often depends on previous words. RNNs model this dependency

- **Handling Variable-Length Input**

- **Learning Temporal/Sequential Patterns**

- In tasks like **language modeling**, **speech recognition**, or **stock prediction**, RNNs learn how the current element relates to past elements over time.

Why Use RNNs?

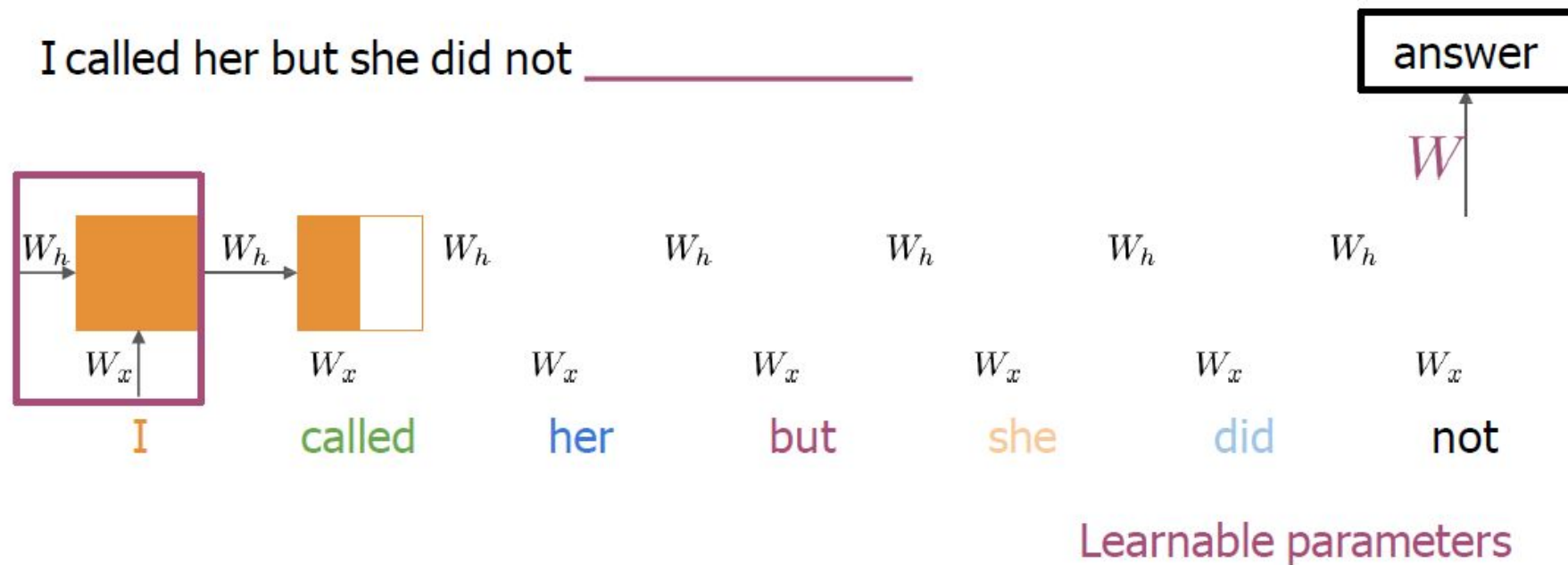
Nour was supposed to study with me. I called her but she **did not** answer

want
respond
choose
want
have
ask
attempt
answer
know

RNNs look at every previous word

Similar probabilities with trigram

RNNs Basic Structure



RNN: Architecture

RNN Architecture: Unrolled View

Unrolled RNN:

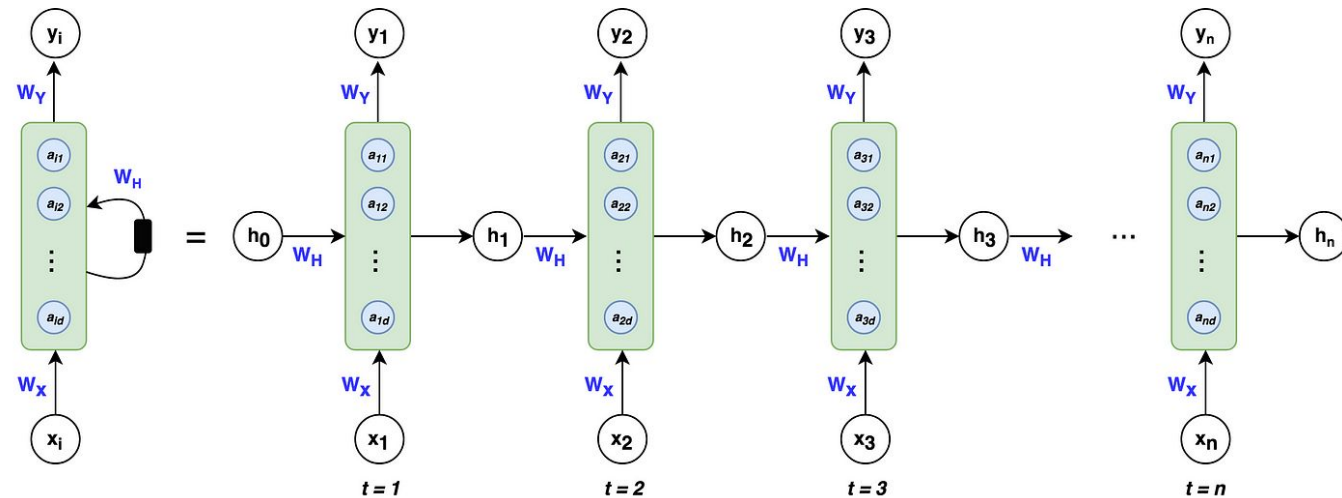
An RNN is essentially a chain of repeating neural network modules, one for each time step.

Each time step shares the same parameters:

- ▶ Input x_t
- ▶ Hidden state h_t
- ▶ Output y_t

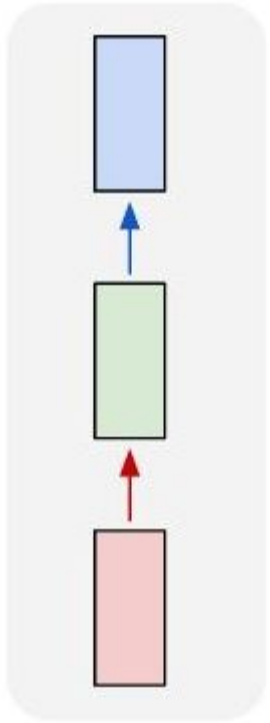
Key Idea

Temporal representation without increasing parameter count!



“Vanilla” Neural Network

one to one



Vanilla Neural Networks

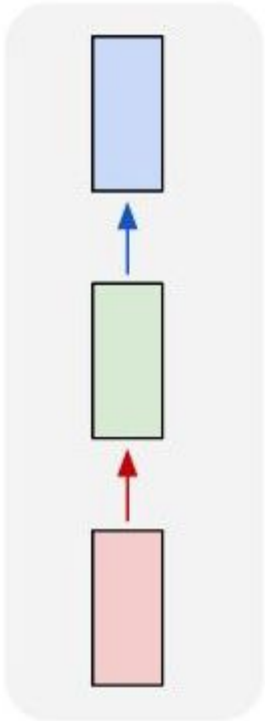
Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

One to One

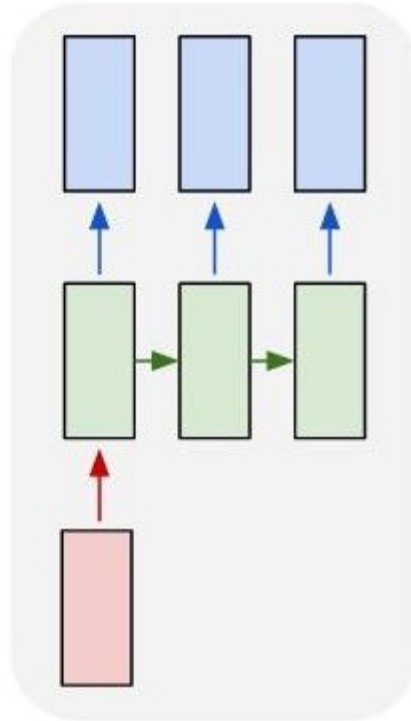


RNN: Process Sequences

one to one

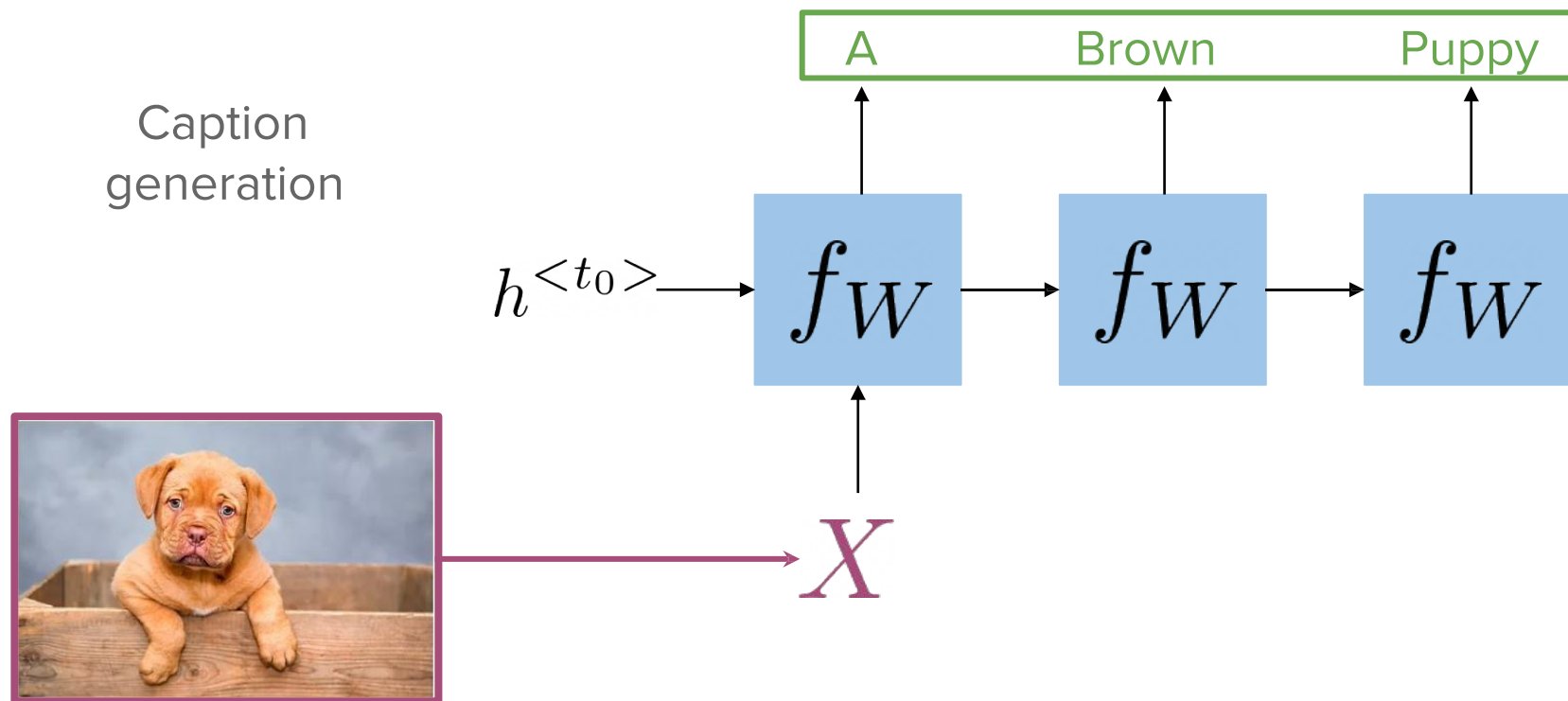


one to many



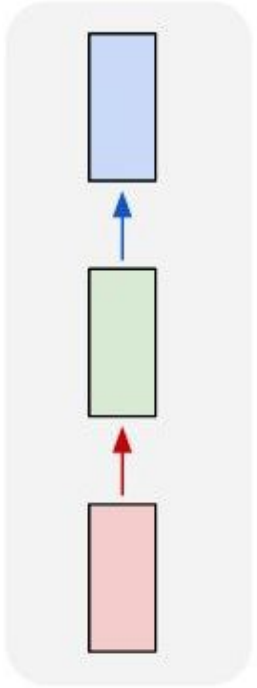
e.g. Image Captioning
image → sequence of words

One to Many

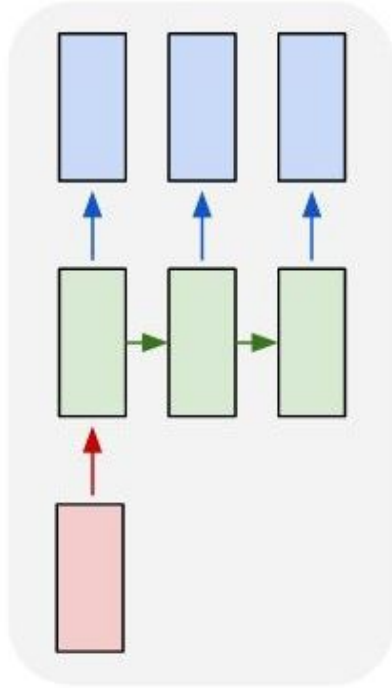


RNN: Process Sequences

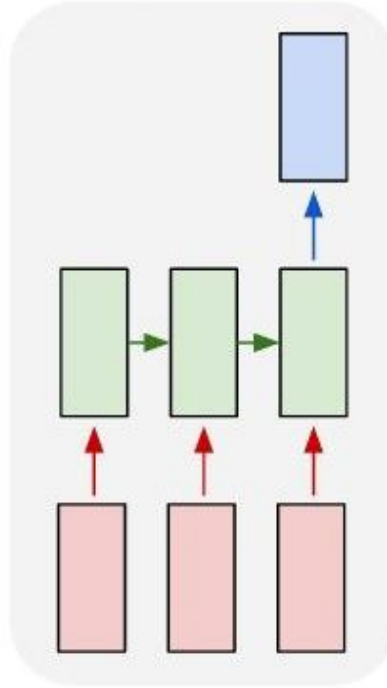
one to one



one to many



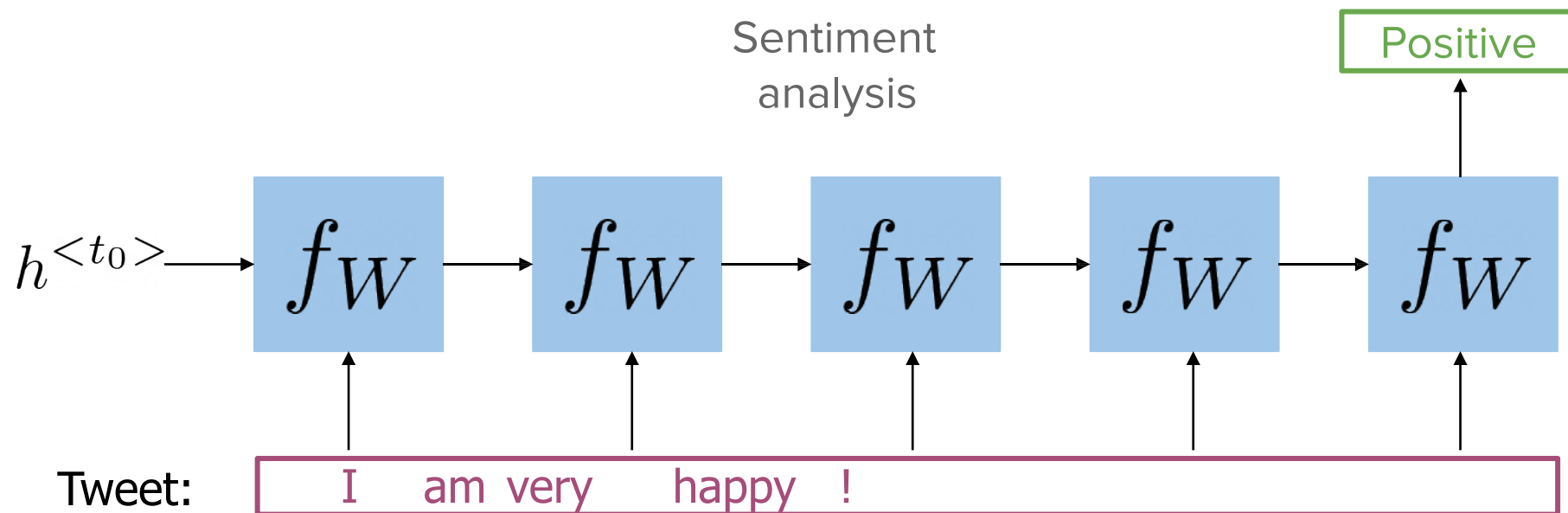
many to one



e.g. Sentiment Analysis
sequence of words → sentiment

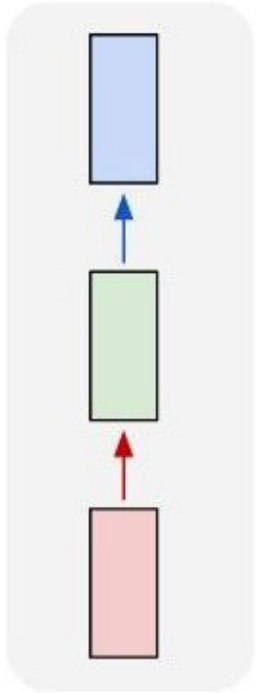
e.g. Action Prediction
sequence of video frames → action class

Many to One

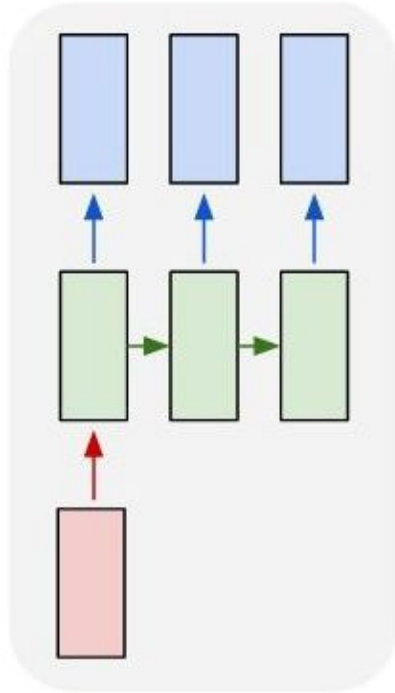


RNN: Process Sequences

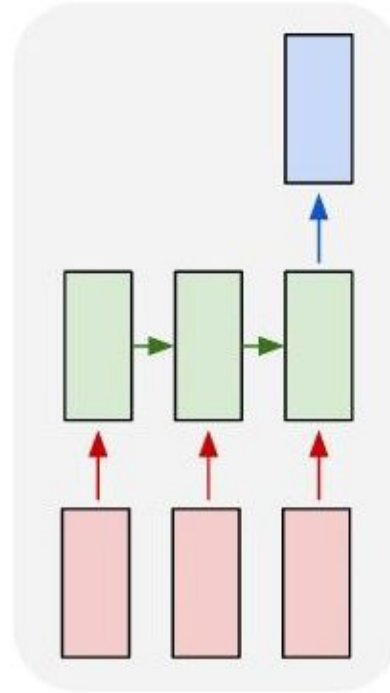
one to one



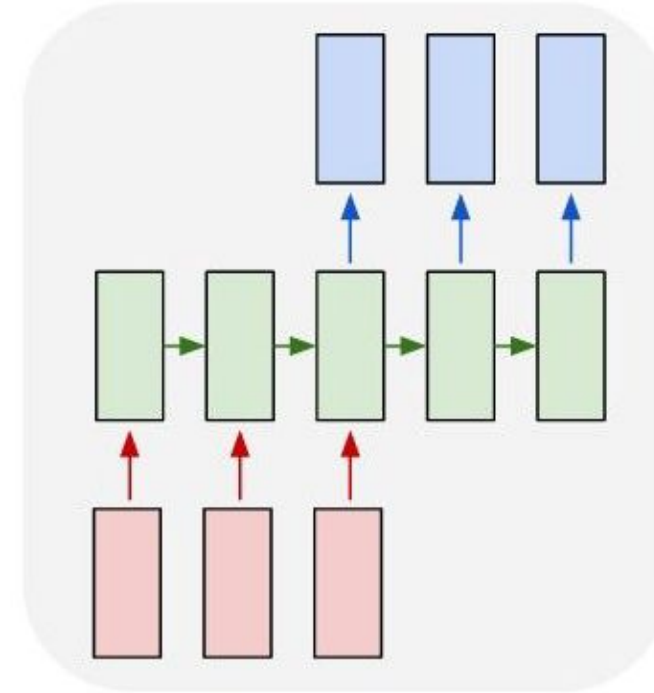
one to many



many to one



many to many



e.g. Video Captioning

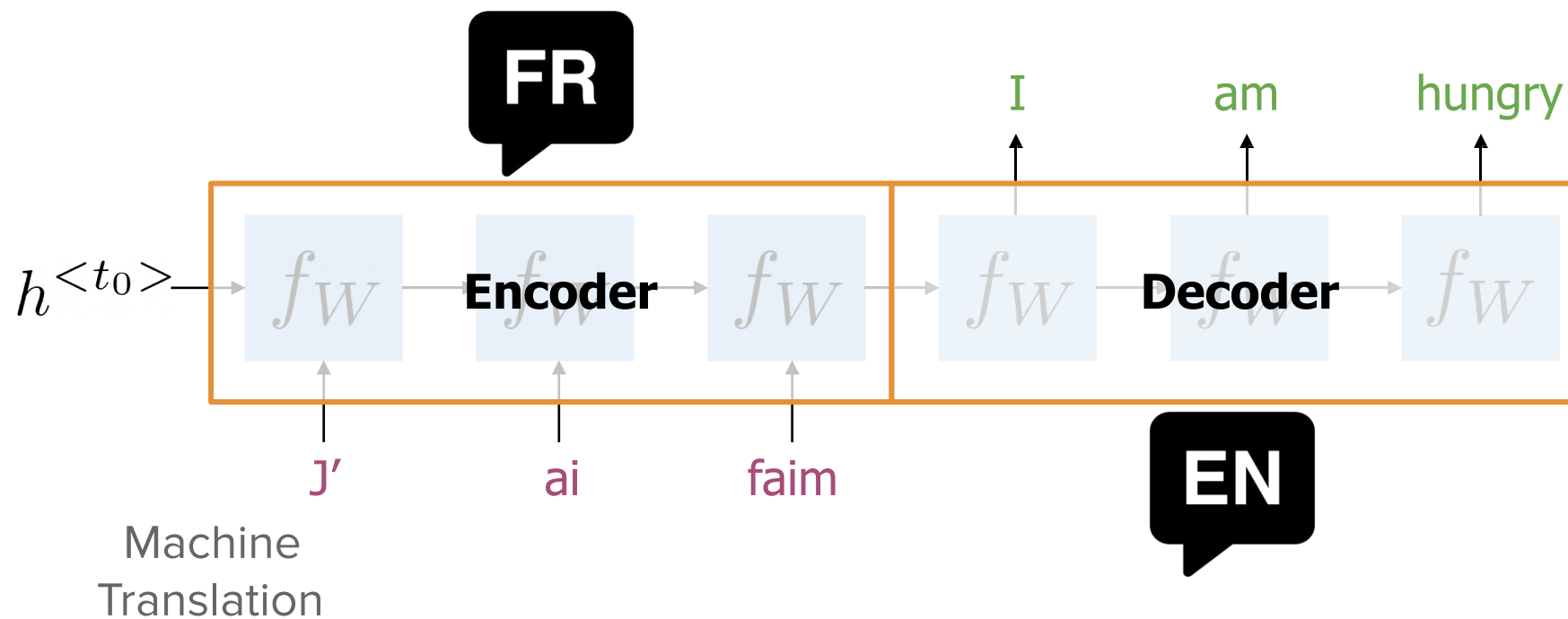
sequence of video frames → caption

e.g. Machine Translation

sequence of words (English) →
sequence of words (French)

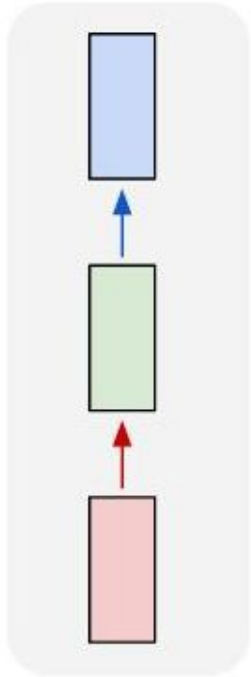
Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Many to Many

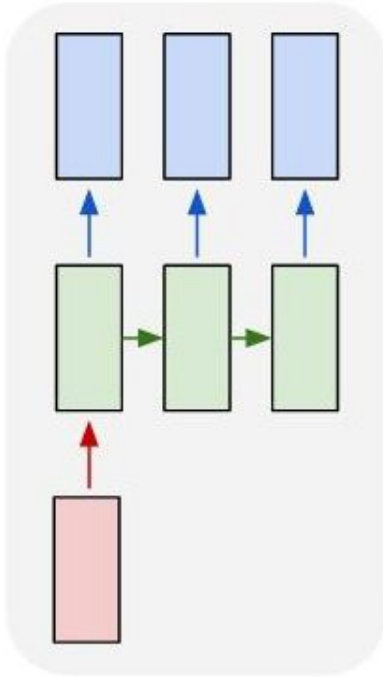


RNN: Process Sequences

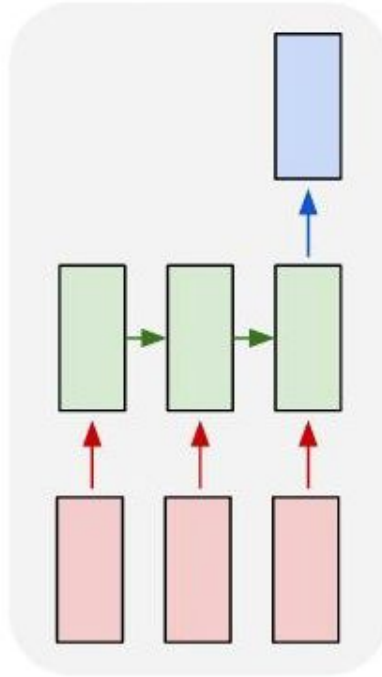
one to one



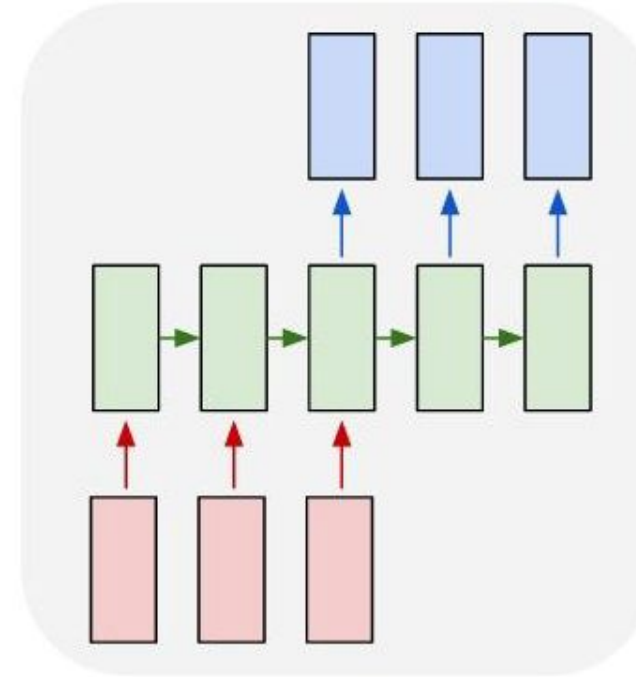
one to many



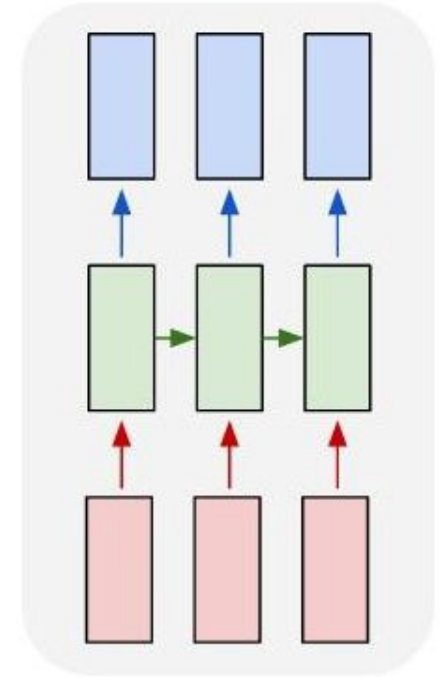
many to one



many to many

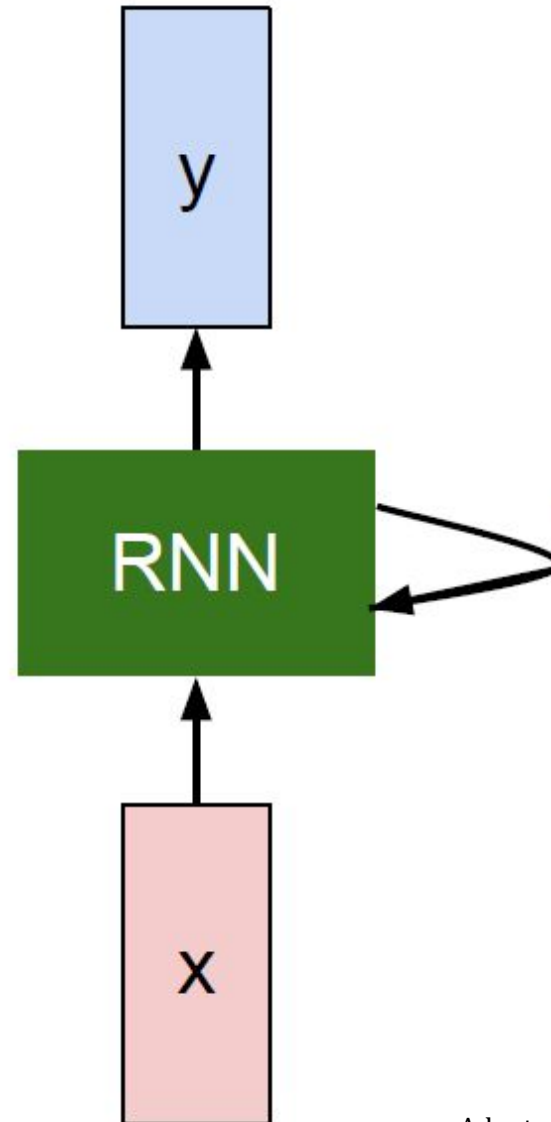


many to many



e.g. Video classification on frame level

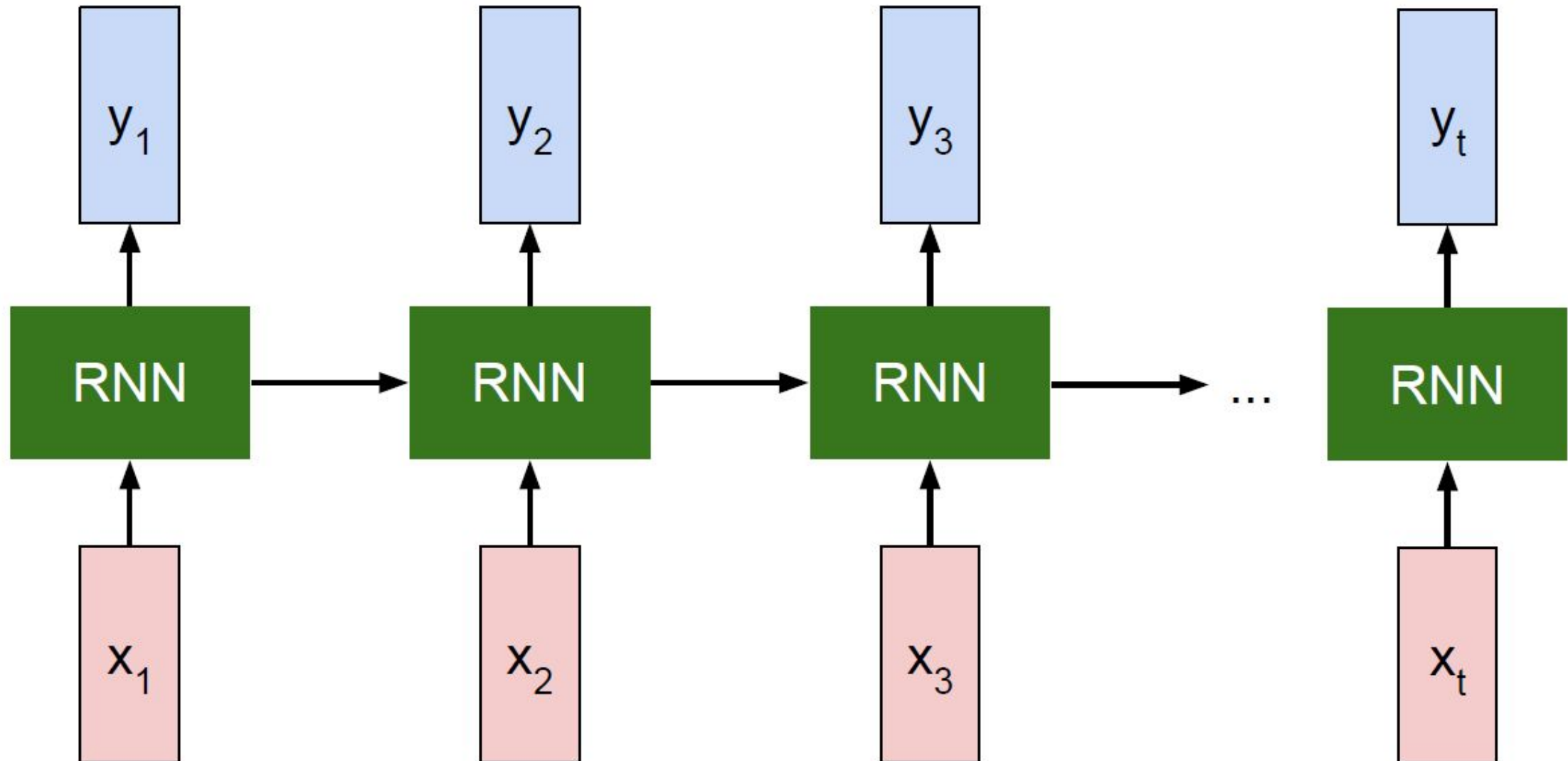
Recurrent Neural Network



Key idea: RNNs have an “internal state” that is updated as a sequence is processed

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Unrolled RNN

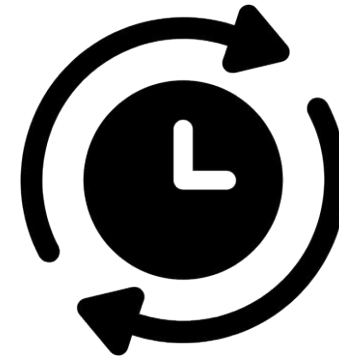


Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

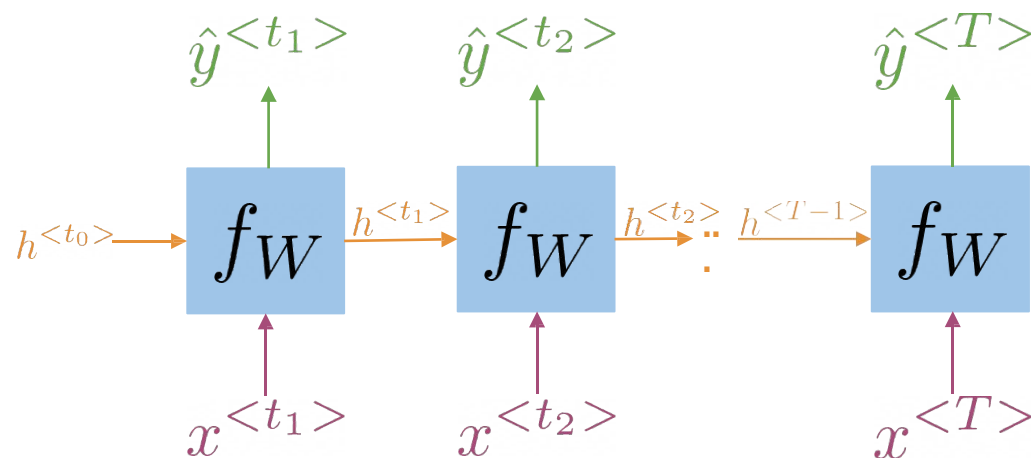
RNN: Mathematical Formulation

Math in Simple RNNs

- How RNNs propagate information (Through time!)
- How RNNs make predictions



A Vanilla RNN

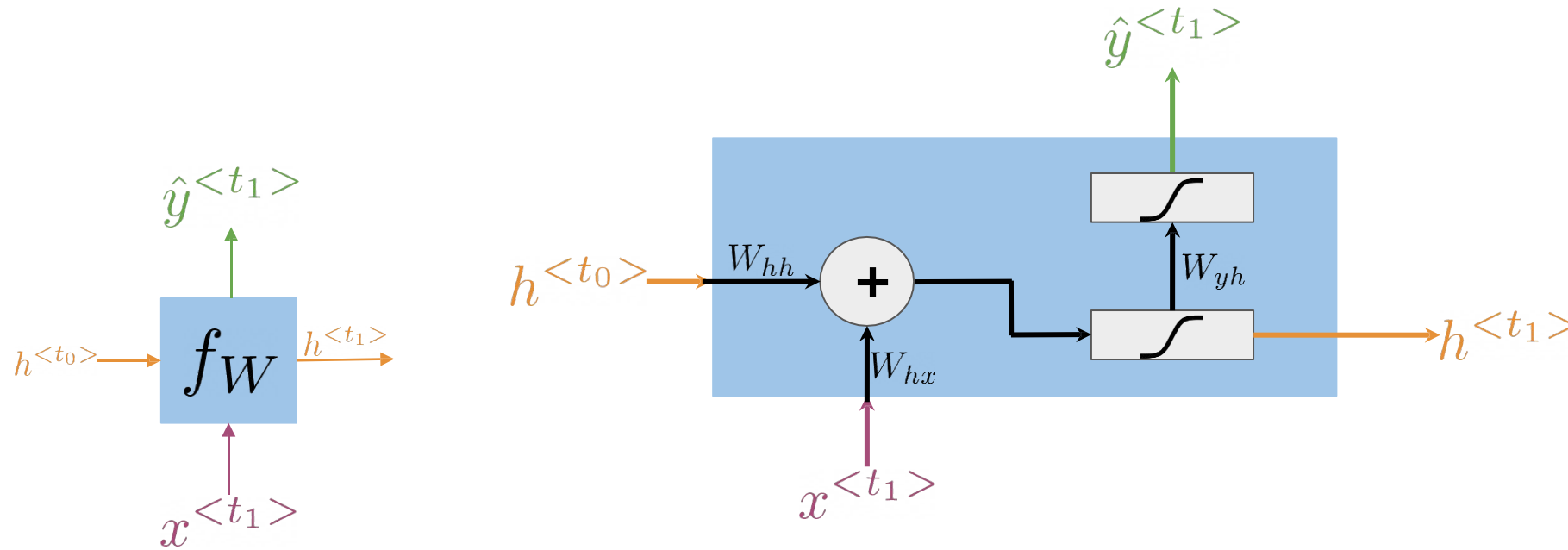


$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$

$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$

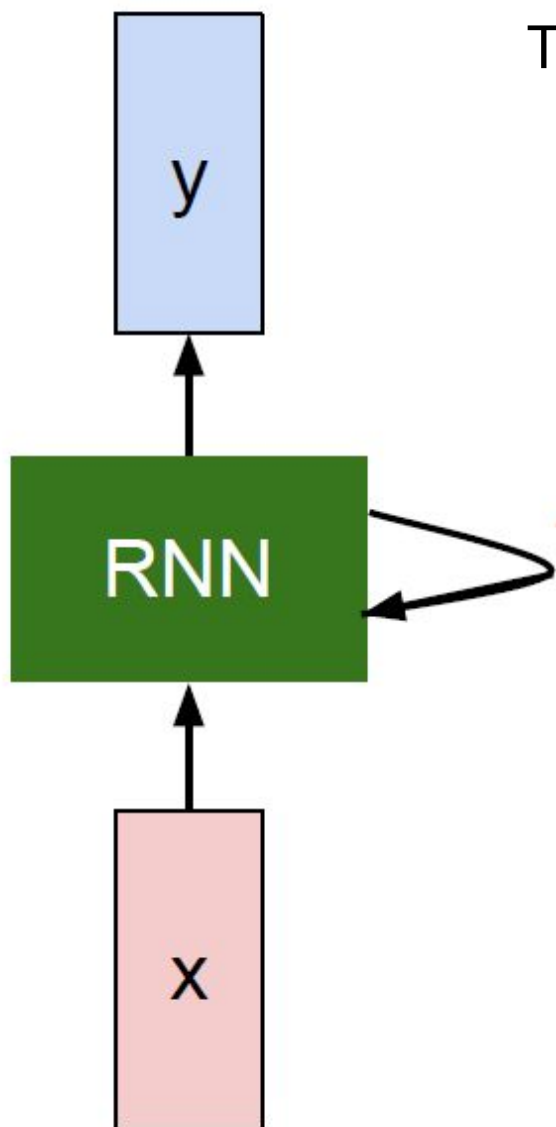
$$h^{<t>} = g(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b_h)$$

A Vanilla RNN



$$h^{<t>} = g(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b_h)$$
$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$

A Vanilla RNN



The state consists of a single “hidden” vector $\mathbf{h}$:

$$h_t = f_W(h_{t-1}, x_t)$$



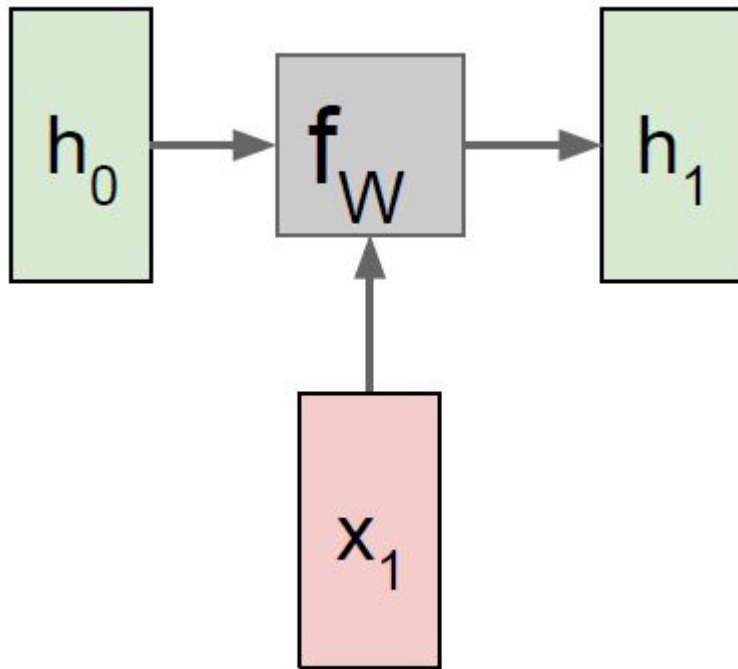
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman

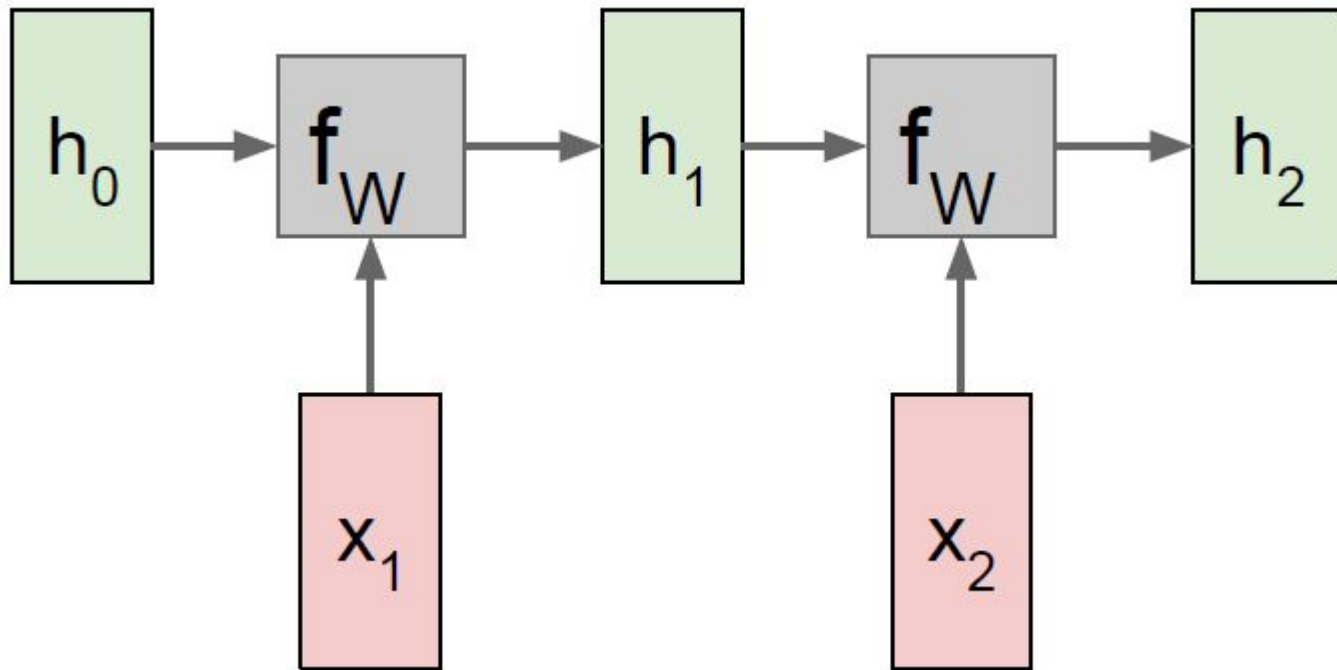
RNN: Parameters Sharing

RNN: Computational Graph



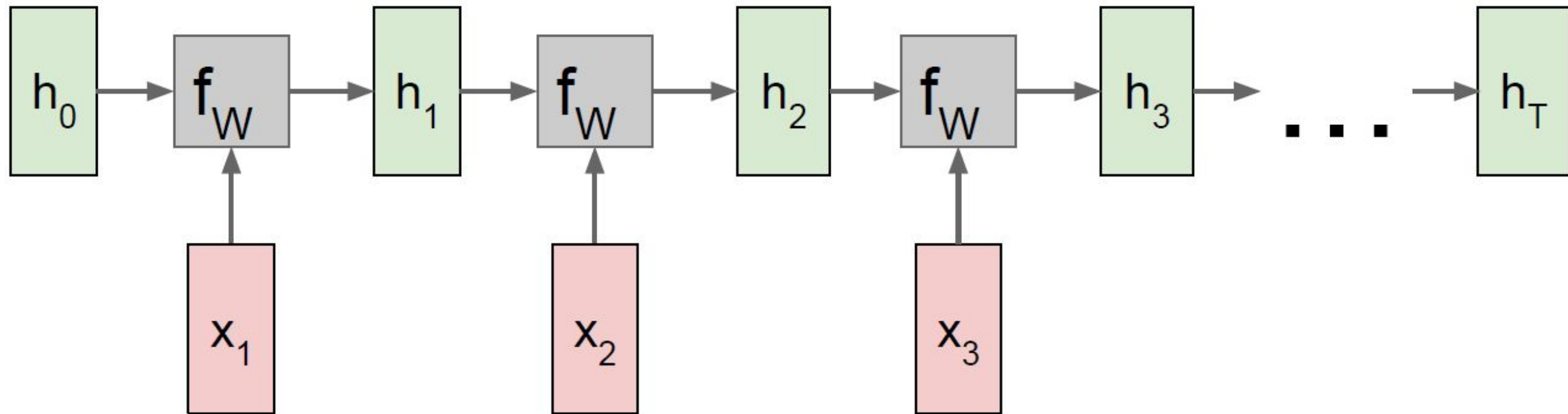
Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

RNN: Computational Graph



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

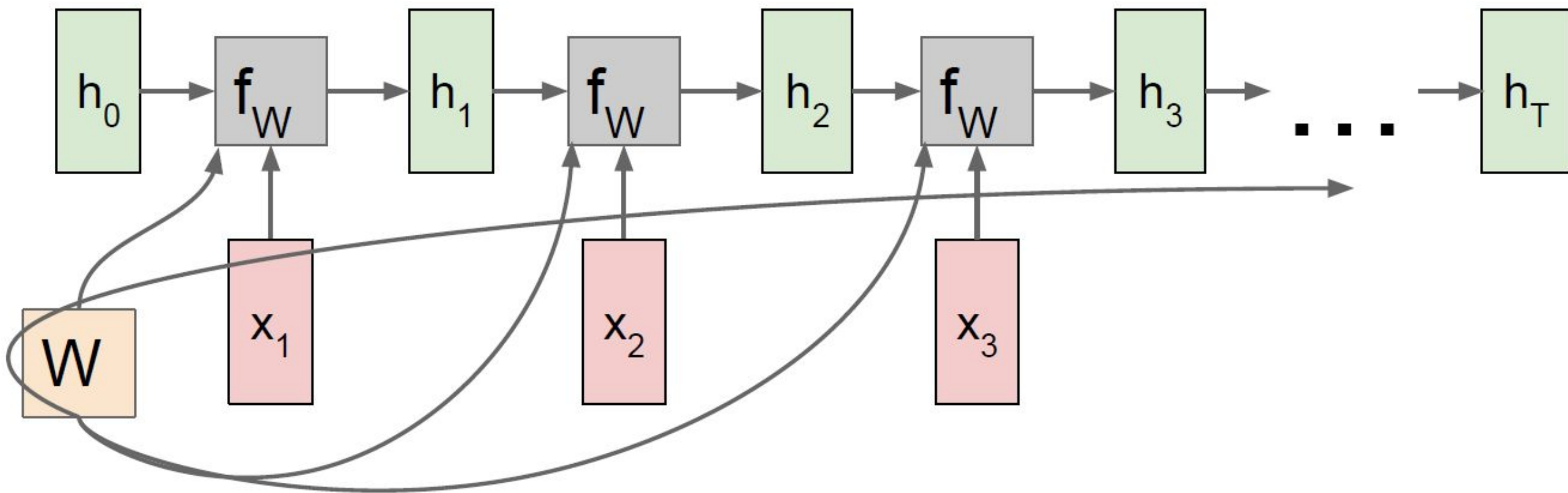
RNN: Computational Graph



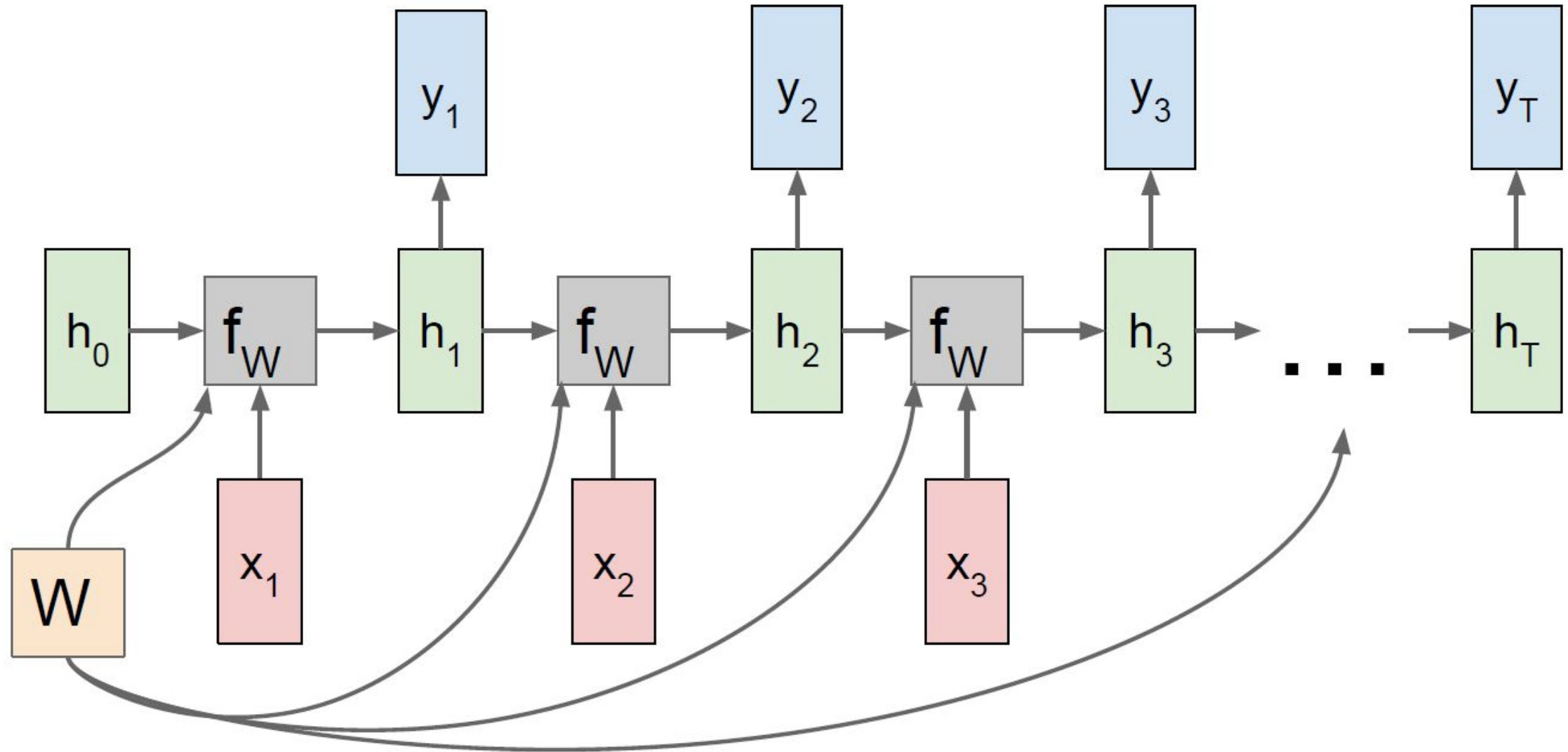
Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

RNN: Computational Graph

Re-use the same weight matrix at every time-step



RNN: Computational Graph - Many to Many



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

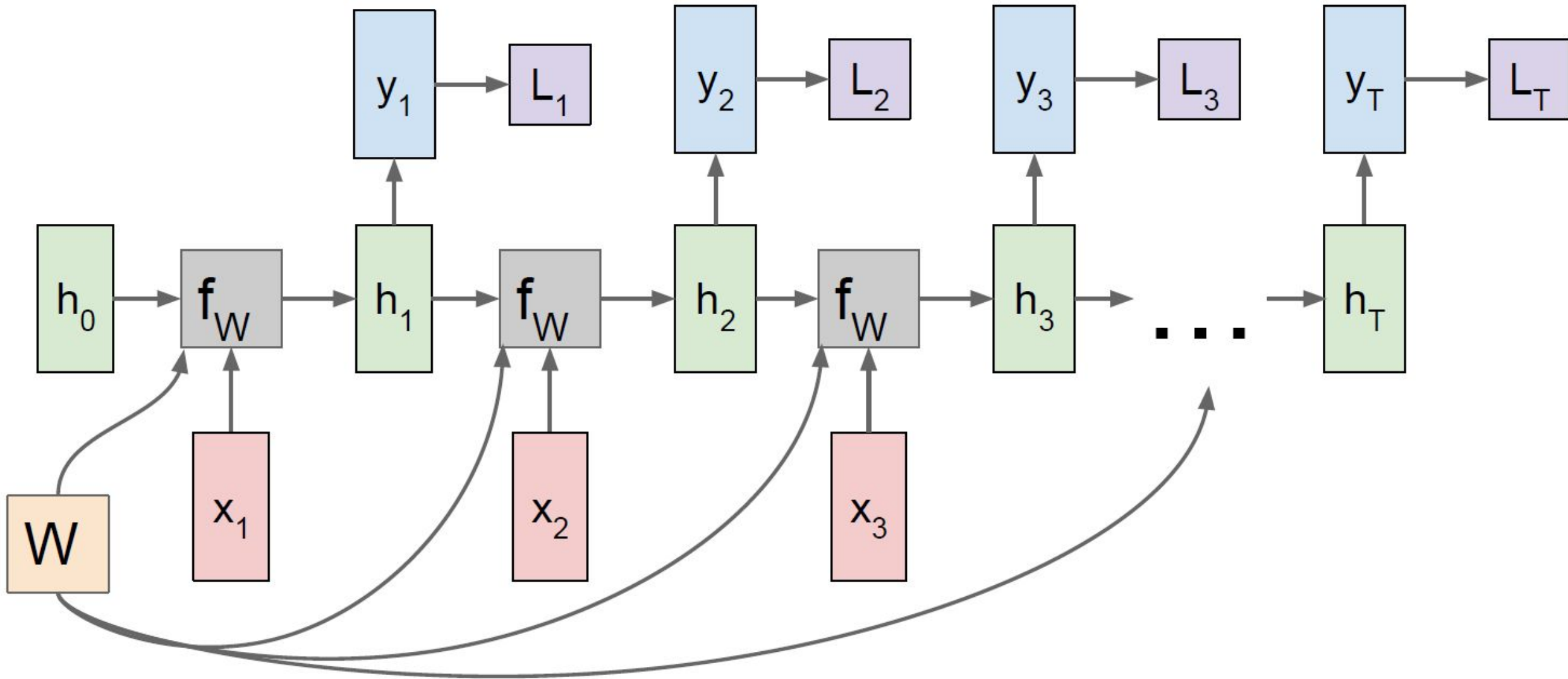
RNN: Computational Graph - Many to Many

أكاديمية كاوست
KAUST ACADEMY



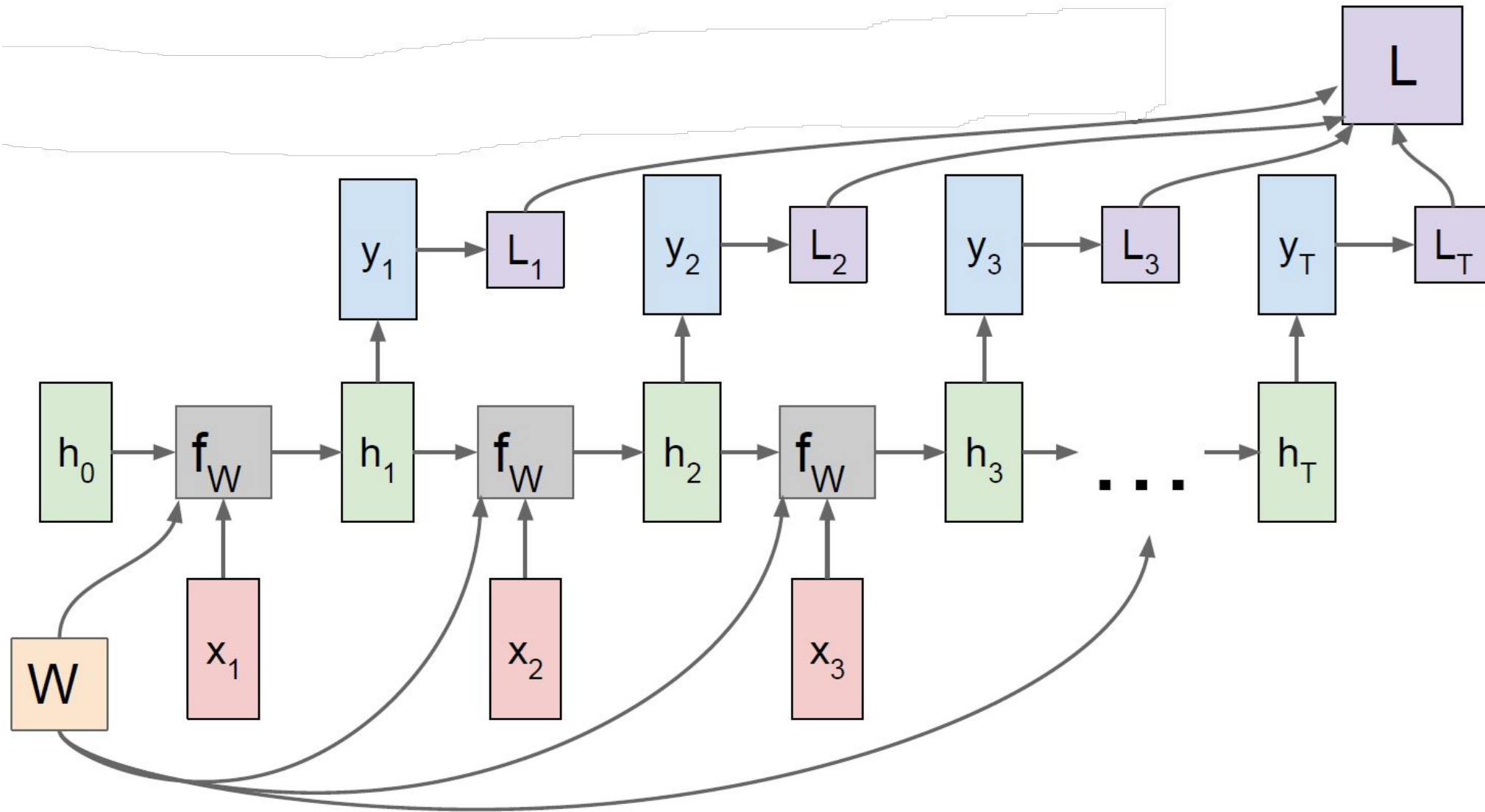
LMH

Lady Margaret Hall



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

RNN: Computational Graph - Many to Many

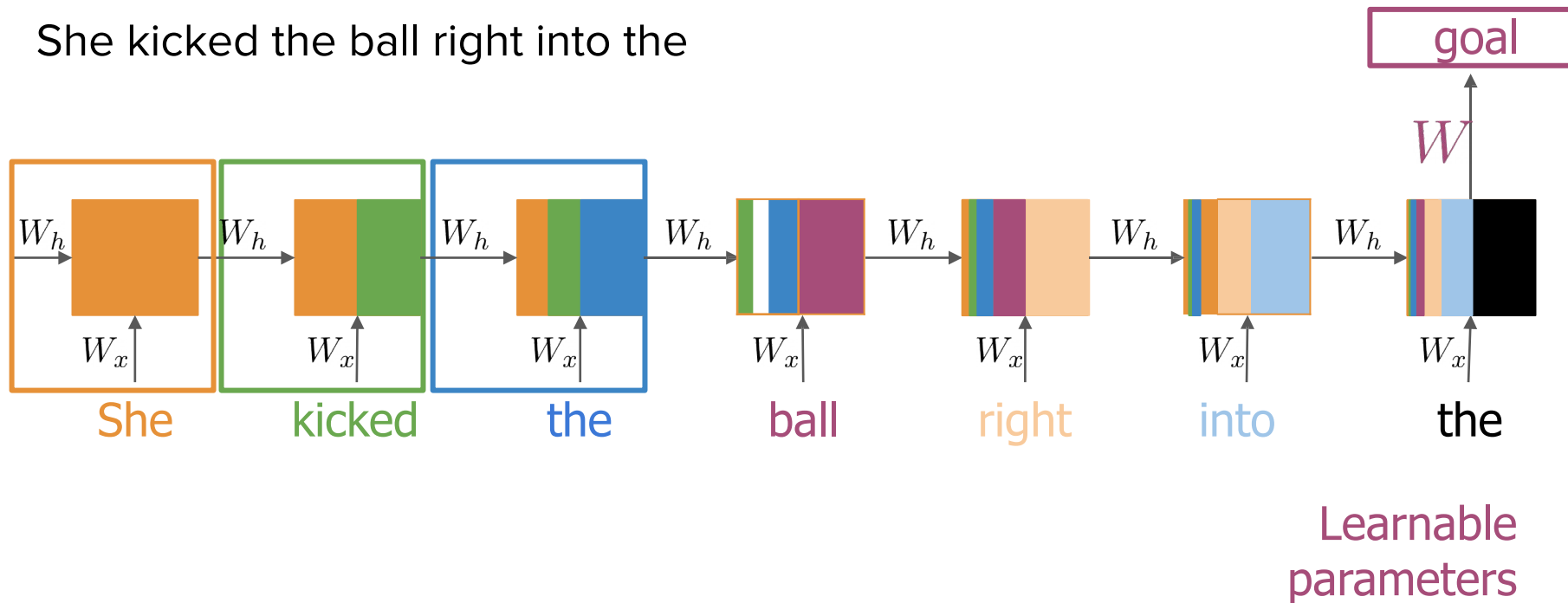


Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

RNN: Challenges

RNN Basic Structure

She kicked the ball right into the

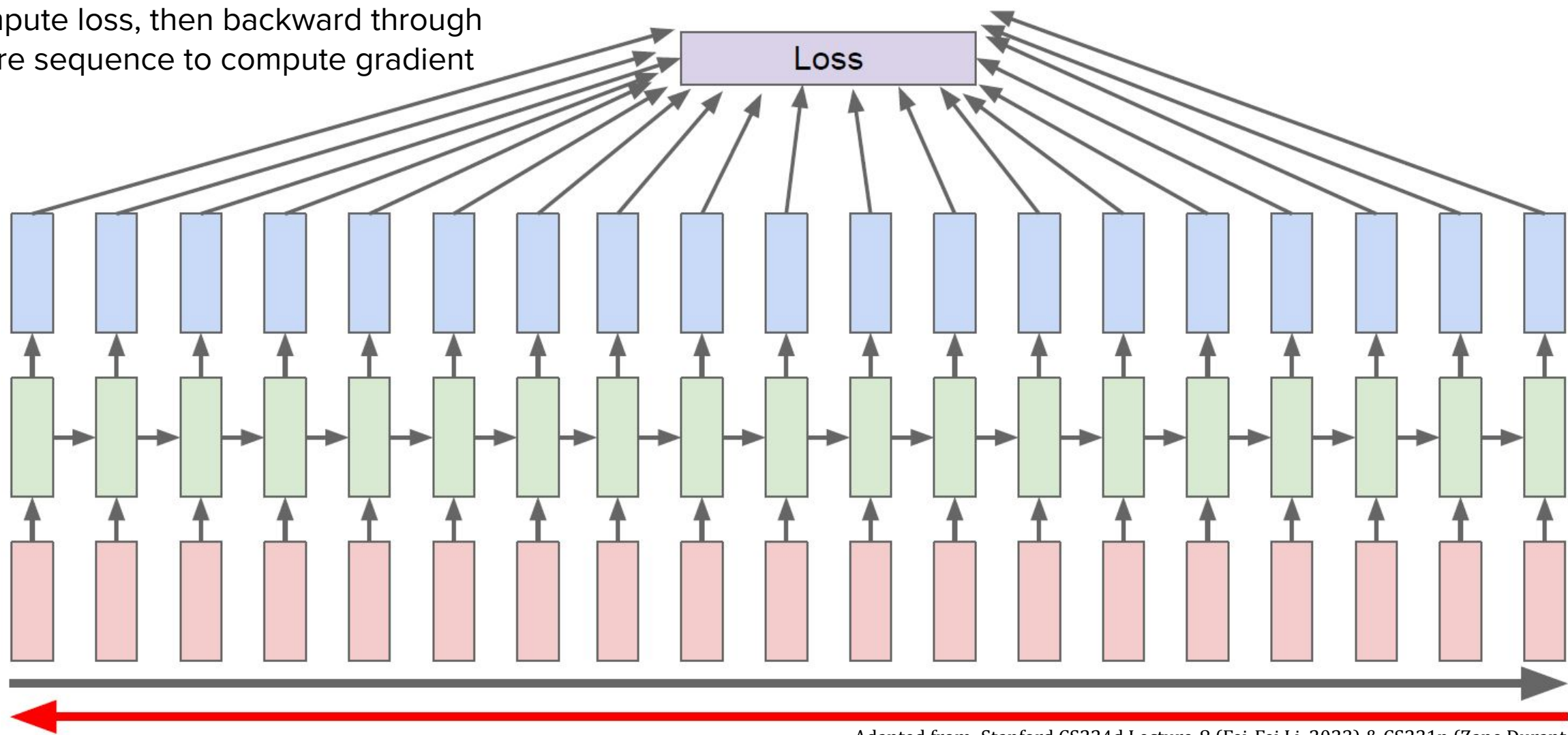


Backpropagation Through Time (BPTT):

- ▶ Training RNNs involves **unfolding** the network across time steps.
- ▶ Standard backpropagation is applied through this unrolled structure.

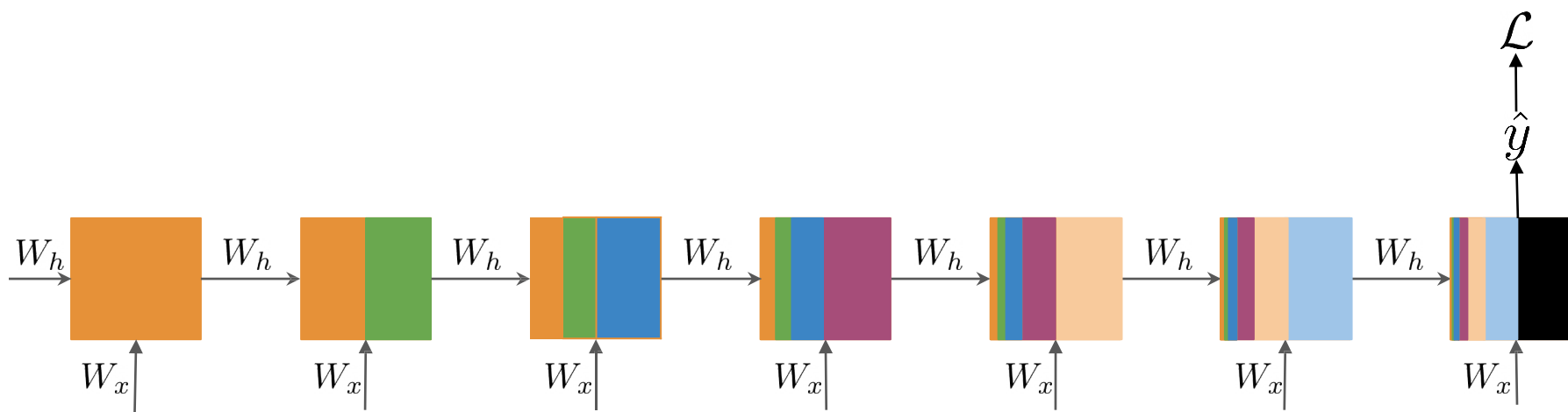
RNN: Backpropagation through time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Backpropagation through time



W_x
 W_h → Same at every step

$$\frac{\partial L}{\partial W_h} \propto \sum_{1 \leq k \leq t} \left(\prod_{t \geq i > k} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W_h}$$

Gradient is proportional to a sum of partial derivative products

Backpropagation through time

$$\frac{\partial L}{\partial W_h} \propto \sum_{1 \leq k \leq t} \left(\prod_{t \geq i > k} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W_h} \rightarrow \text{Contribution of hidden state } k$$

Length of the product proportional to
how far k is from t

$$\frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{\partial h_{t-2}}{\partial h_{t-3}} \frac{\partial h_{t-3}}{\partial h_{t-4}} \frac{\partial h_{t-4}}{\partial h_{t-5}} \frac{\partial h_{t-5}}{\partial h_{t-6}} \frac{\partial h_{t-6}}{\partial h_{t-7}} \frac{\partial h_{t-7}}{\partial h_{t-8}} \frac{\partial h_{t-8}}{\partial h_{t-9}} \frac{\partial h_{t-9}}{\partial h_{t-10}} \frac{\partial h_{t-10}}{\partial W_h}$$

Contribution of hidden state $t-10$

Backpropagation through time

$$\frac{\partial L}{\partial W_h} \propto \sum_{1 \leq k \leq t} \left(\prod_{t \geq i > k} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W_h}$$

Contribution of hidden state k

Length of the product proportional to
how far k is from t

Partial derivatives < 1	Contribution goes to 0	Vanishing Gradient
Partial derivatives > 1	Contribution goes to infinity	Exploding Gradient

Problems:

- ▶ **Vanishing Gradients:** Gradients shrink as they are propagated back, making it hard to learn long-term dependencies.
- ▶ **Exploding Gradients:** Gradients grow exponentially, leading to unstable updates.

Solutions Preview (covered in future modules):

- ▶ Use of LSTM and GRU architectures
- ▶ Gradient Clipping

Vanishing and Exploding Gradients

Backpropagation Through Time (BPTT) spreads gradients across many time steps.

Vanishing Gradients:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \rightarrow 0$$

- ▶ Early layers barely learn
- ▶ Forget long-term dependencies

Exploding Gradients:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \rightarrow \infty$$

- ▶ Unstable updates, diverging weights

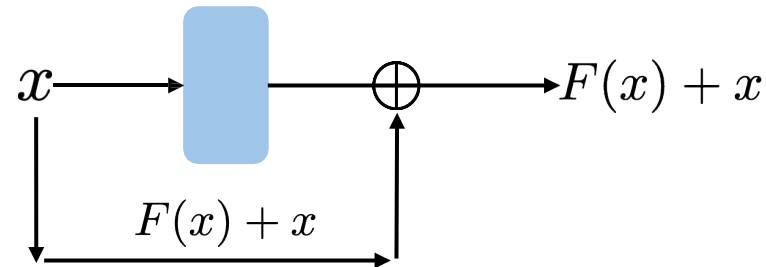


- Identity RNN with ReLU activation

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Gradient clipping

- Skip connections



RNN Advantages:

- Can process any length input
- Computation for step t can (in theory) use information from many steps back
- Model size doesn't increase for longer input
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

LSTM - Long short term memory unit

Introduced by Hochreiter & Schmidhuber (1997)

Core Idea: LSTM uses **gates** to control what to keep, forget, and output.

Key Components:

- ▶ Forget Gate f_t
- ▶ Input Gate i_t
- ▶ Cell State C_t
- ▶ Output Gate o_t

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state

$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state

$$h_t = o \odot \tanh(c_t)$$

Equations:

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$$

$$\tilde{C}_t = \tanh(W_C[x_t, h_{t-1}] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$$

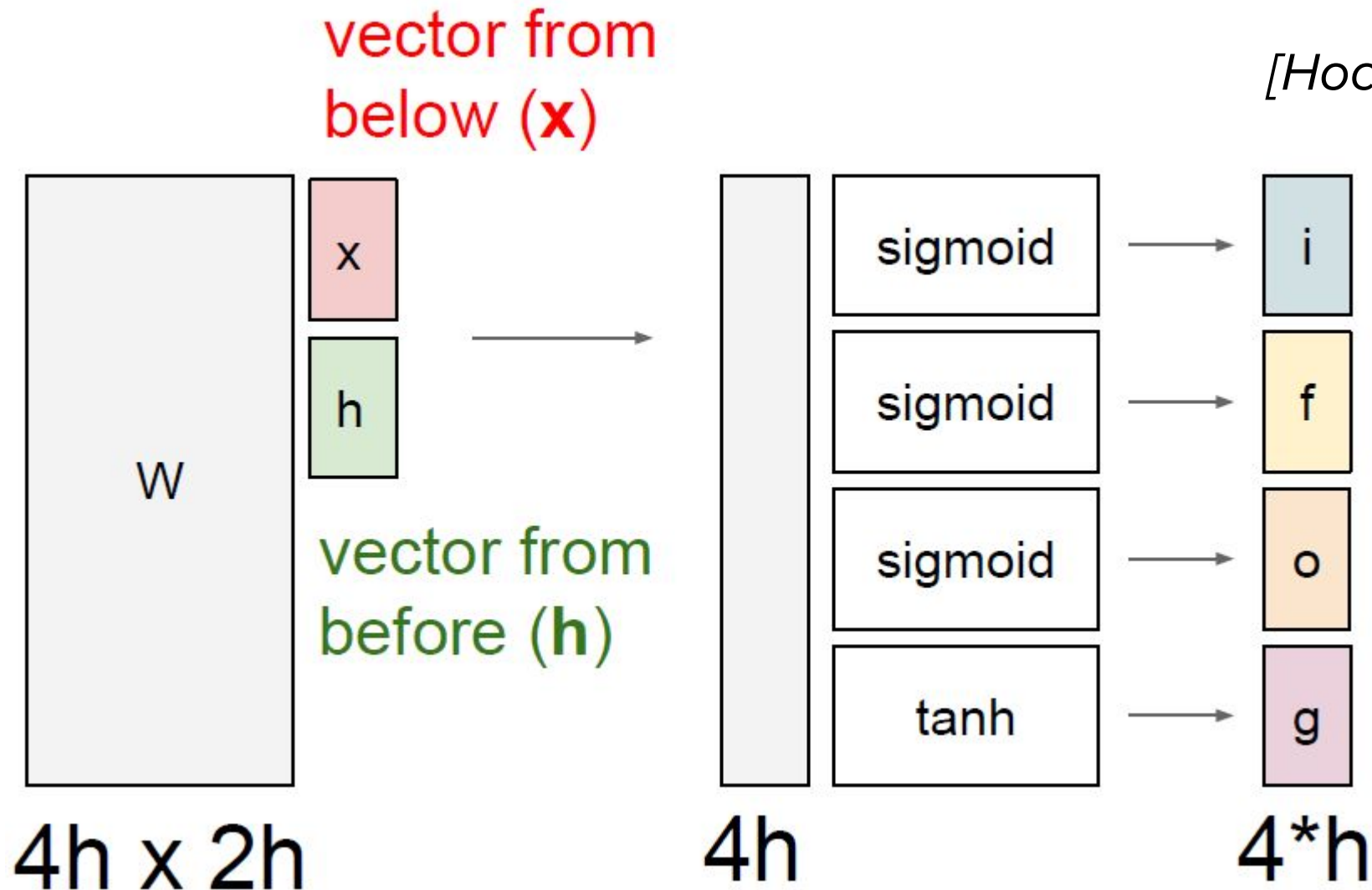
$$h_t = o_t * \tanh(C_t)$$

Helps retain long-term dependencies

LSTMs: a memorable solution

- Learns when to remember and when to forget
- Basic anatomy:
 - A cell state
 - A hidden state
 - Multiple gates
- Gates allow gradients to avoid vanishing and exploding

LSTM – Long Short-Term Memory

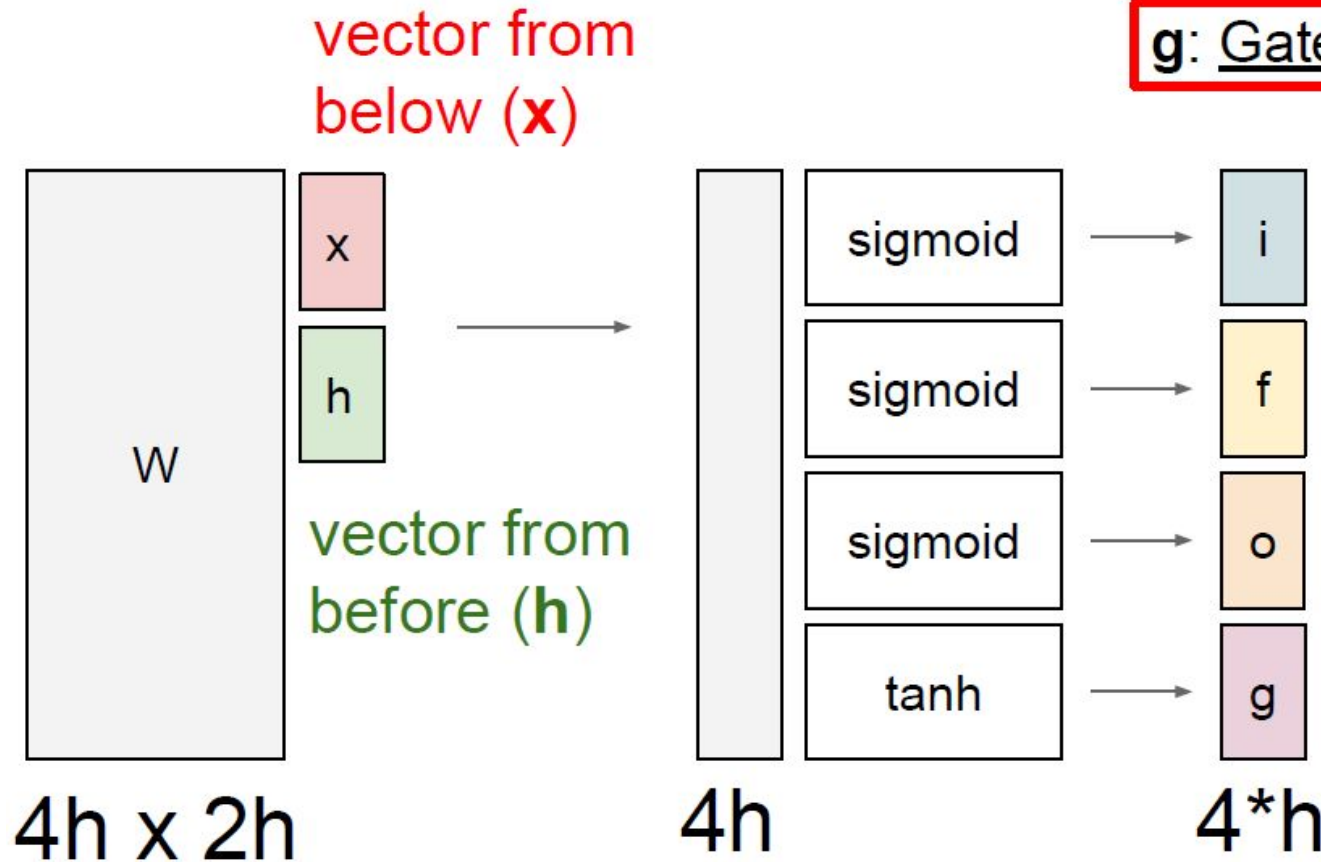


Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

LSTM – Long Short-Term Memory

[Hochreiter et al., 1997]

g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \text{tanh} \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

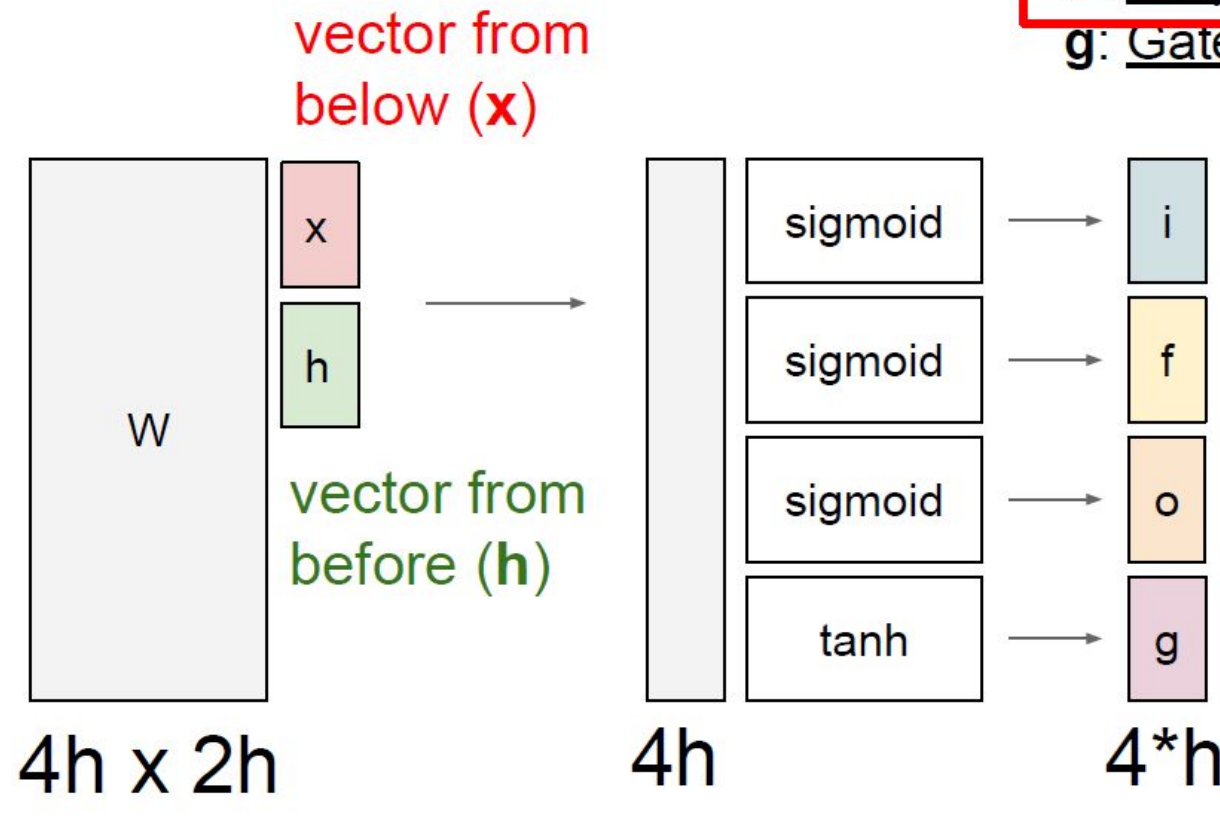
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

LSTM – Long Short-Term Memory

[Hochreiter et al., 1997]

- i: Input gate, whether to write to cell
- f: Forget gate. Whether to erase cell
- o: Output gate, How much to reveal cell**
- g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

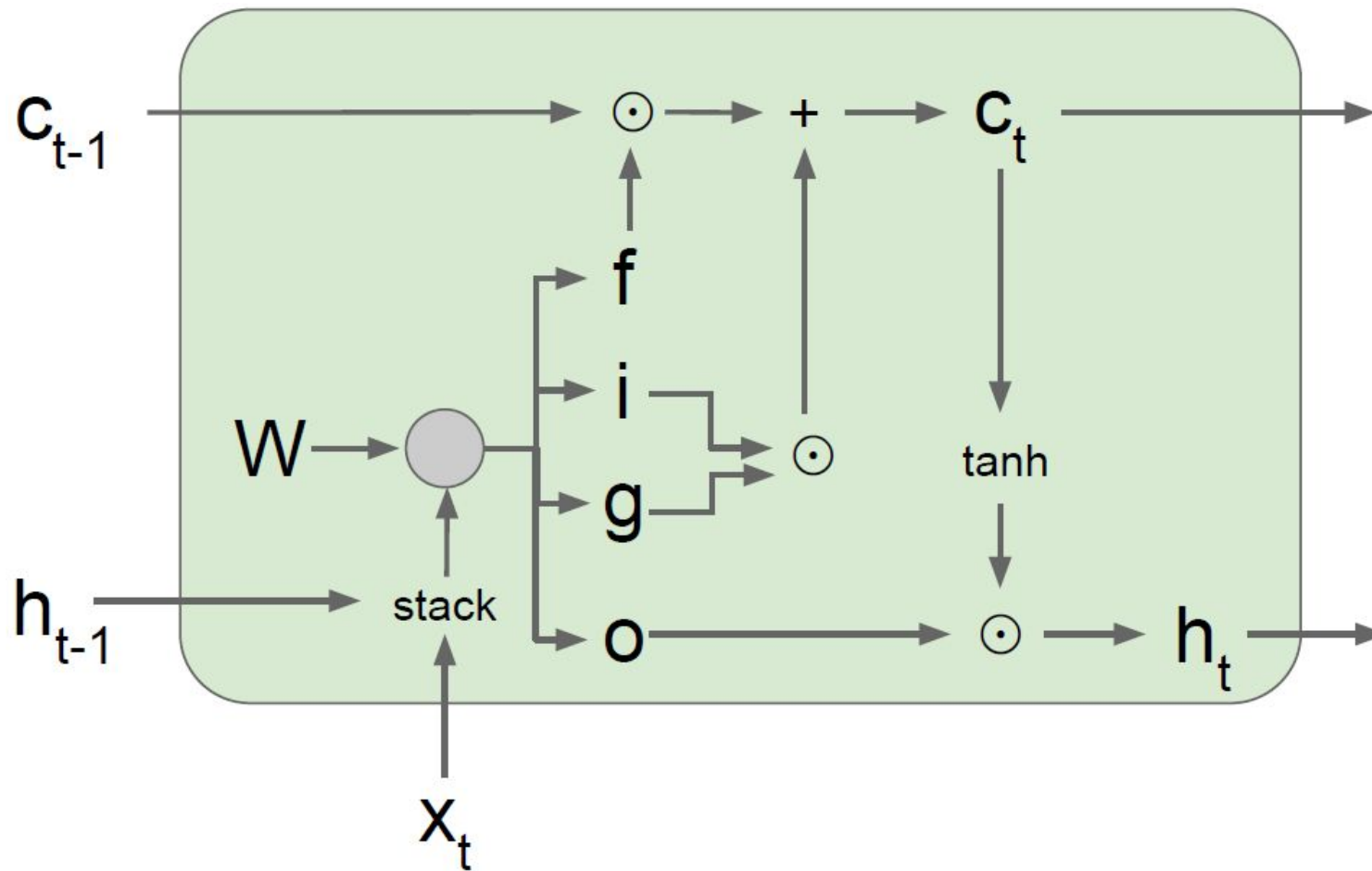
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = \mathbf{o} \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

LSTM – Long Short-Term Memory

[Hochreiter et al., 1997]



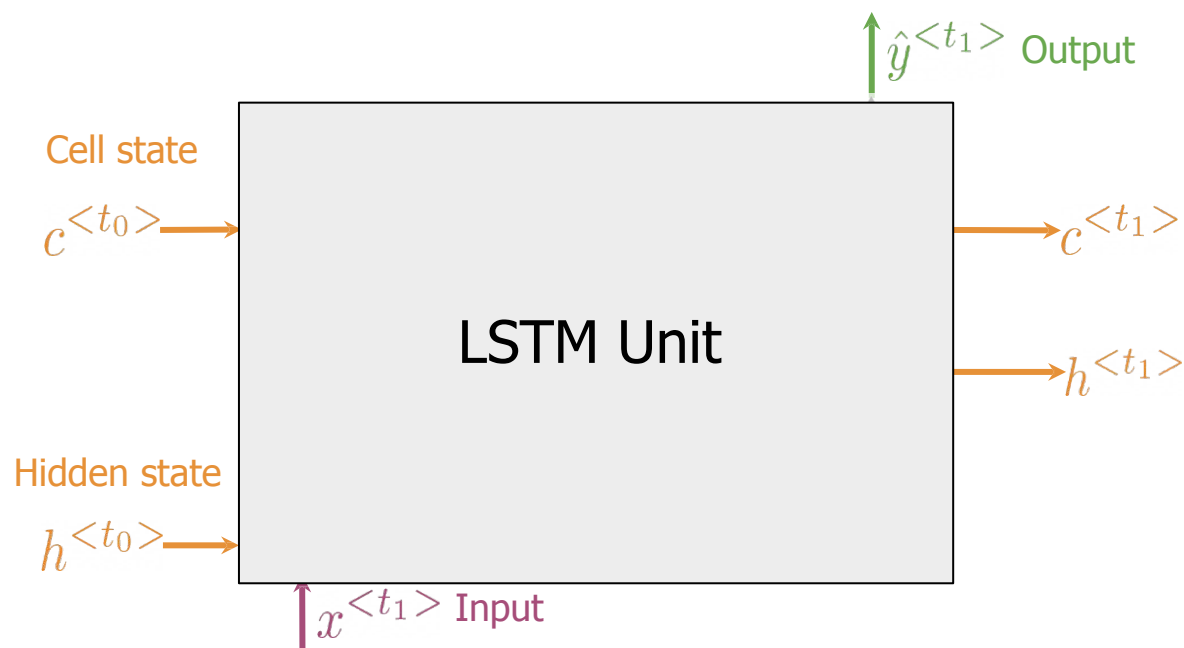
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Gates in LSTM

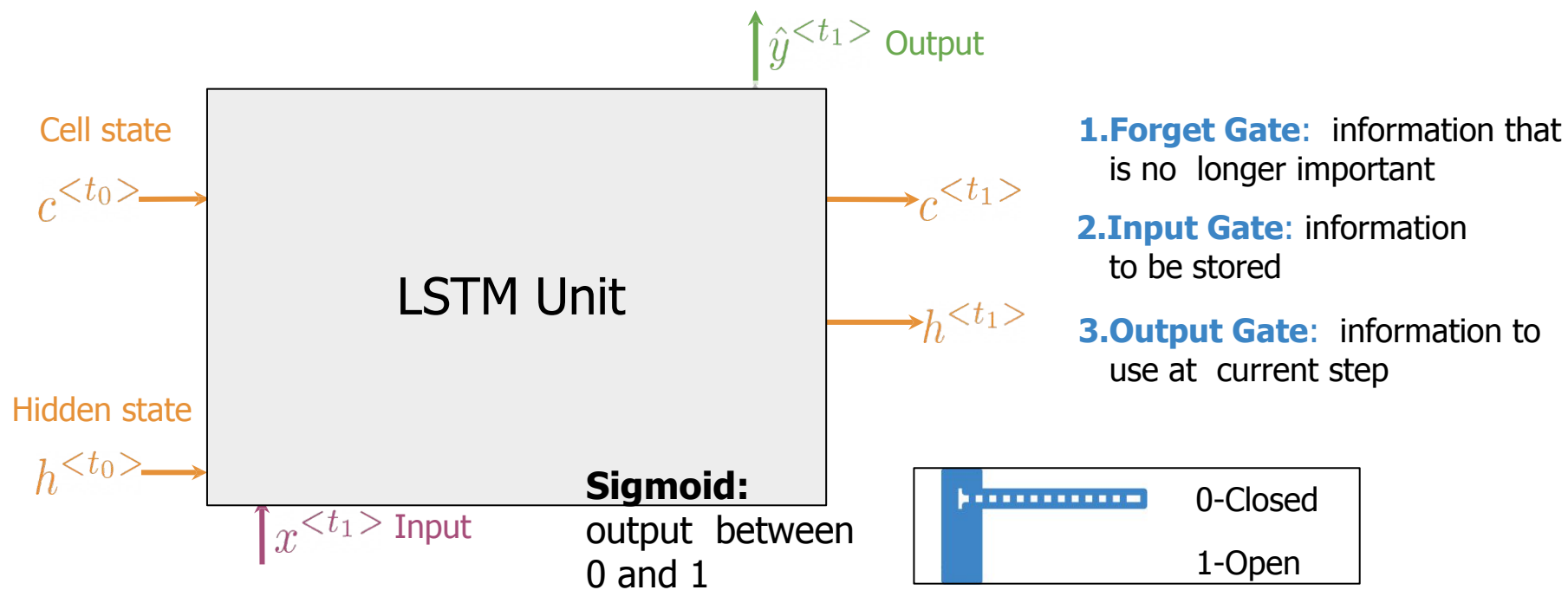


1. Forget Gate: information that is no longer important

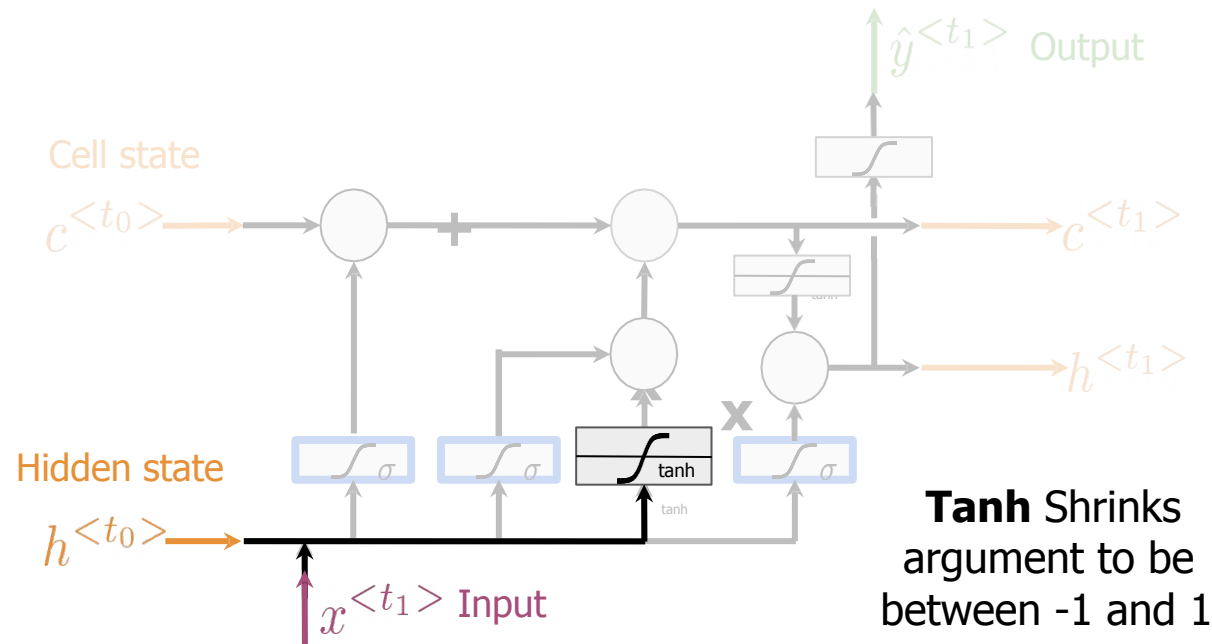
2. Input Gate: information to be stored

3. Output Gate: information to use at current step

Gates in LSTM



Candidate Cell State

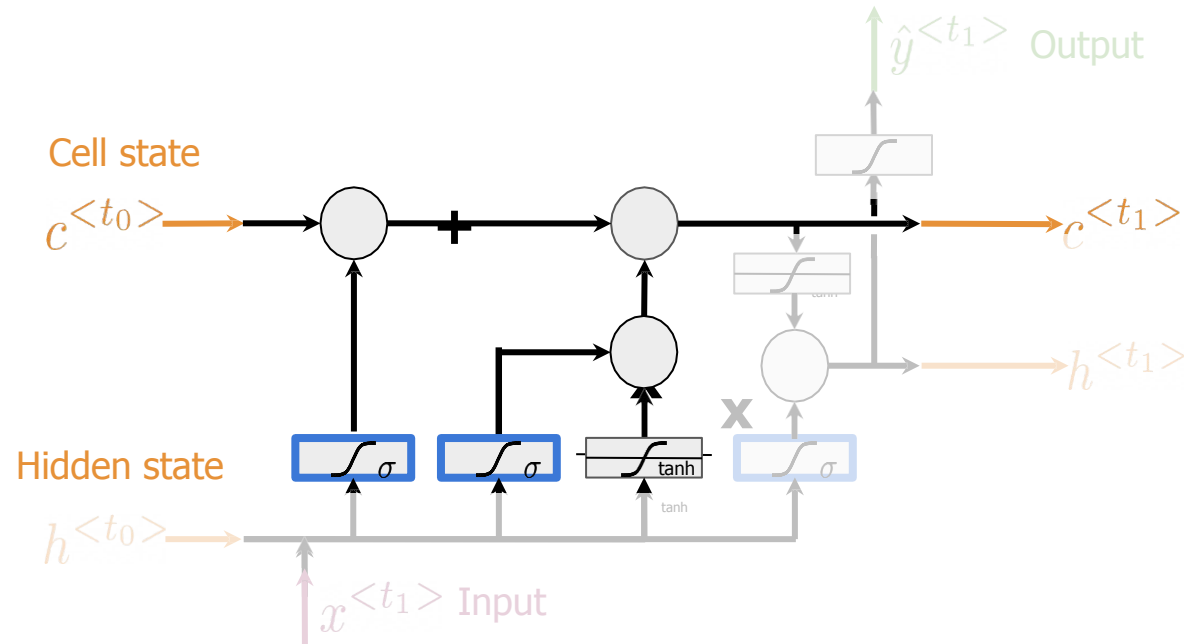


Candidate cell state

Information from the previous **hidden state** and current **input**

Other activations could be used

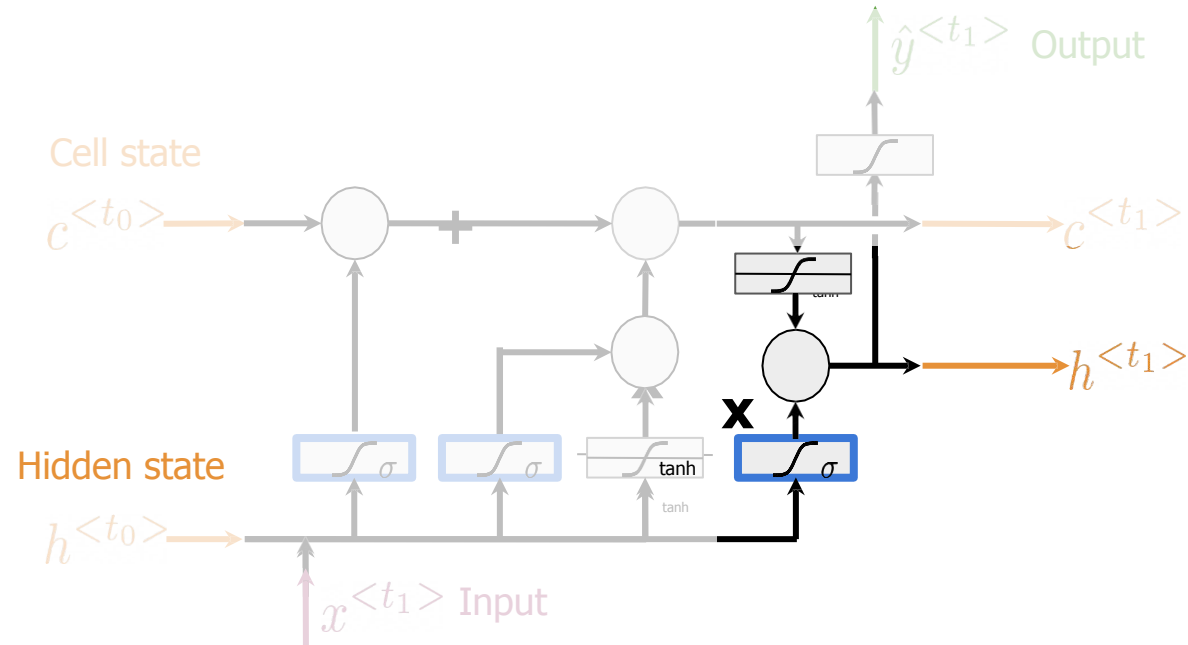
New Cell State



New Cell state

Add information from the **candidate cell state** using the **forget** and **input gates**

New Hidden State



New Hidden State

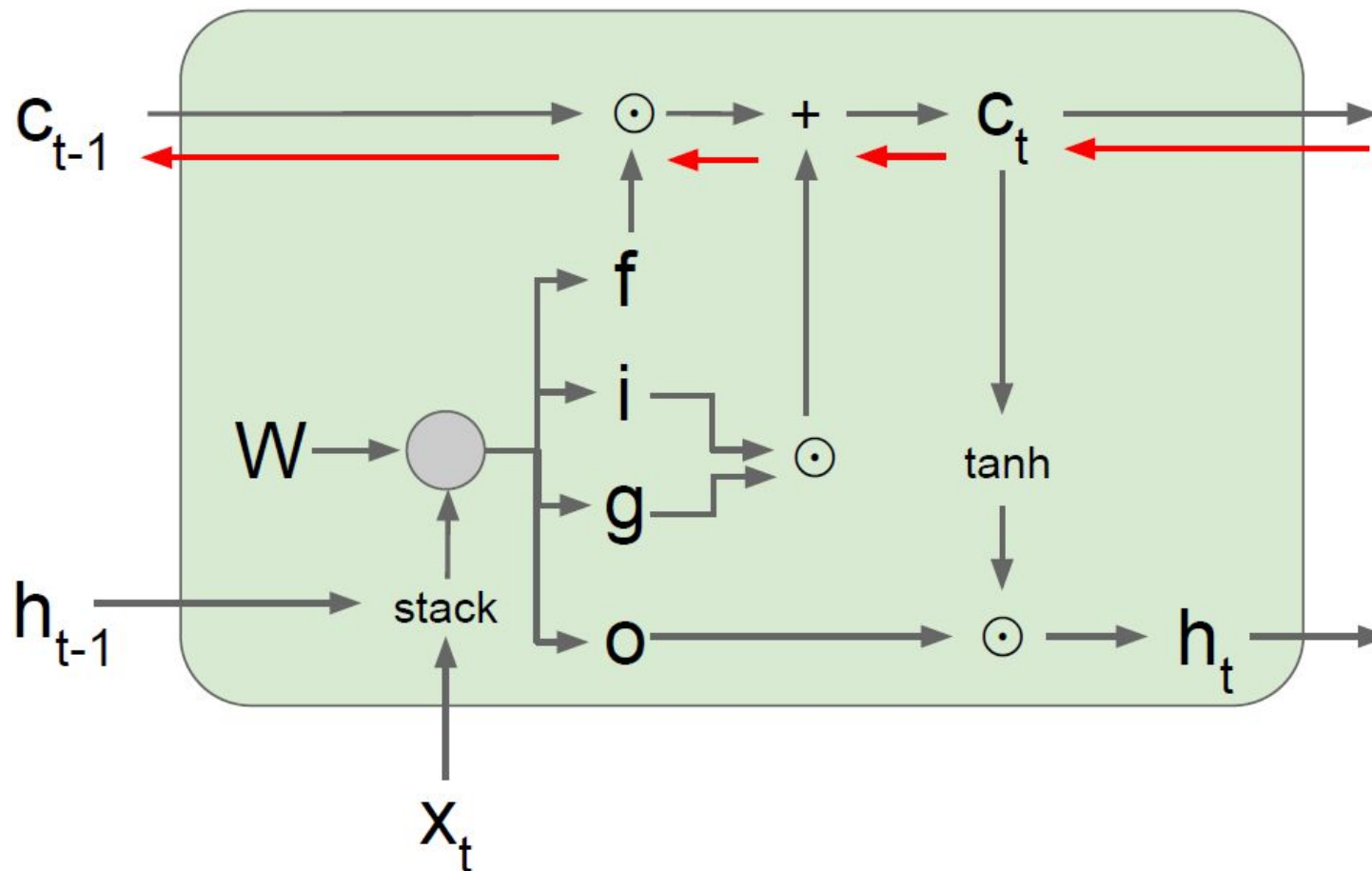
Select information from the **new cell state** using the **output gate**

The **Tanh** activation could be omitted

LSTM: Gradient Flow

[Hochreiter et al., 1997]

Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W



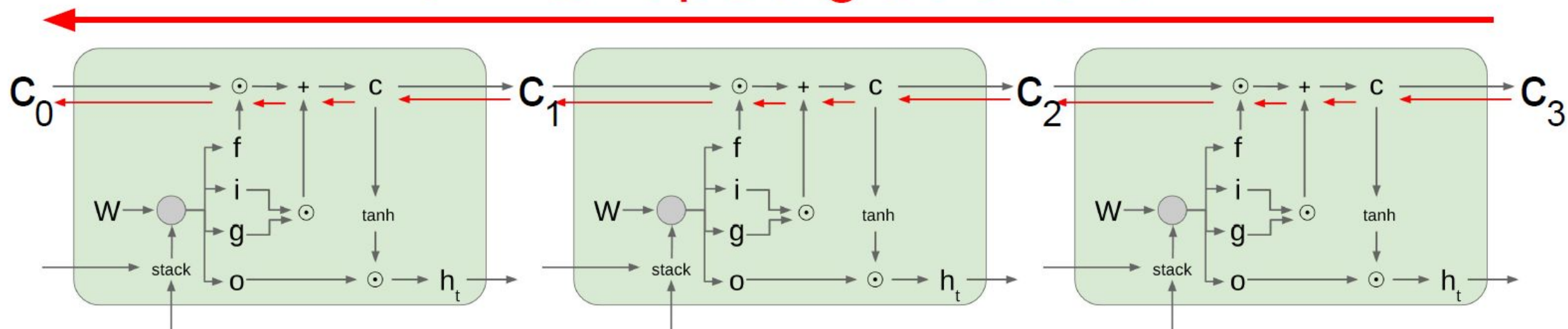
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

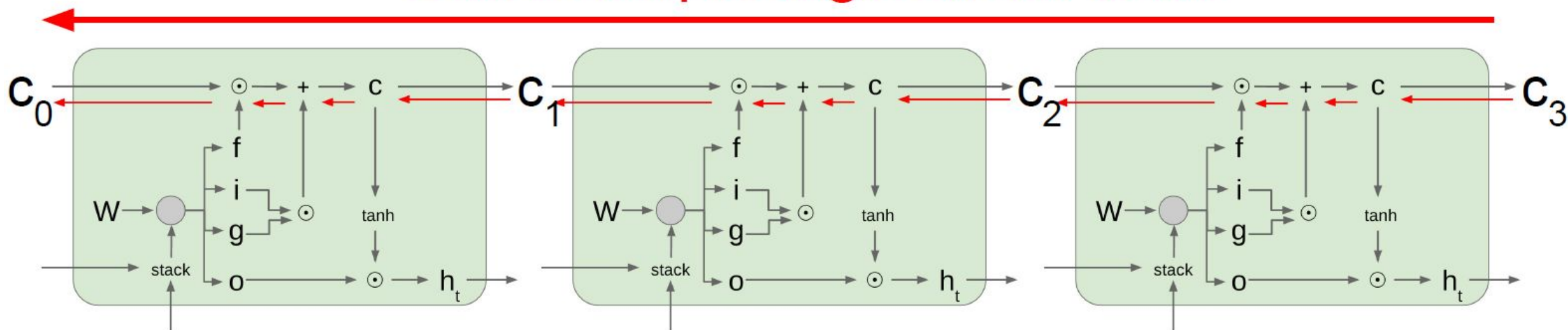
[Hochreiter et al., 1997]

Uninterrupted gradient flow!

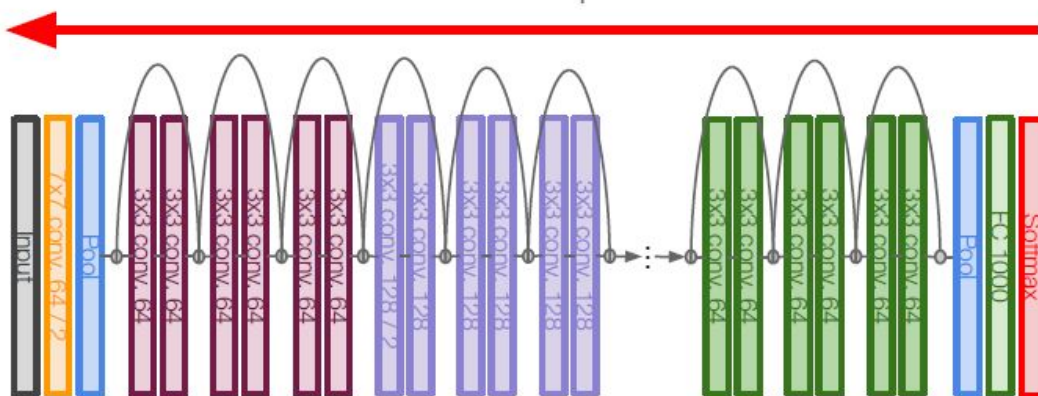


[Hochreiter et al., 1997]

Uninterrupted gradient flow!



Similar to ResNet!



In between:
Highway Networks

$$g = T(x, W_T)$$

$$y = g \odot H(x, W_H) + (1 - g) \odot x$$

Srivastava et al, "Highway Networks",
ICML DL Workshop 2015

Do LSTMs solve the vanishing gradient problem?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
 - e.g. if the $f = 1$ and the $i = 0$, then the information of that cell is preserved indefinitely.
 - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state
- LSTM **doesn't guarantee** that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

GRU - Gated Recurrent Unit

Introduced by Cho et al., 2014

Simpler than LSTM, with fewer gates

- ▶ No separate memory cell
- ▶ Combines forget and input into **update gate**

Key Components:

- ▶ Update Gate z_t
- ▶ Reset Gate r_t

Equations:

$$z_t = \sigma(W_z[x_t, h_{t-1}])$$

$$r_t = \sigma(W_r[x_t, h_{t-1}])$$

$$\tilde{h}_t = \tanh(W_h[x_t, r_t * h_{t-1}])$$

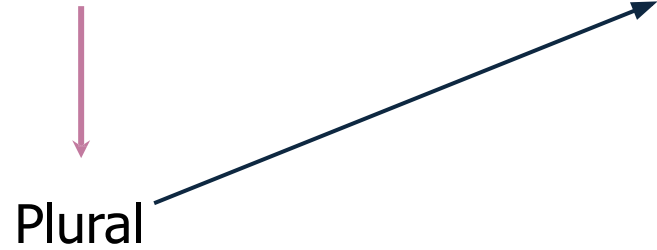
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Comparable performance to LSTM

Faster training, fewer parameters

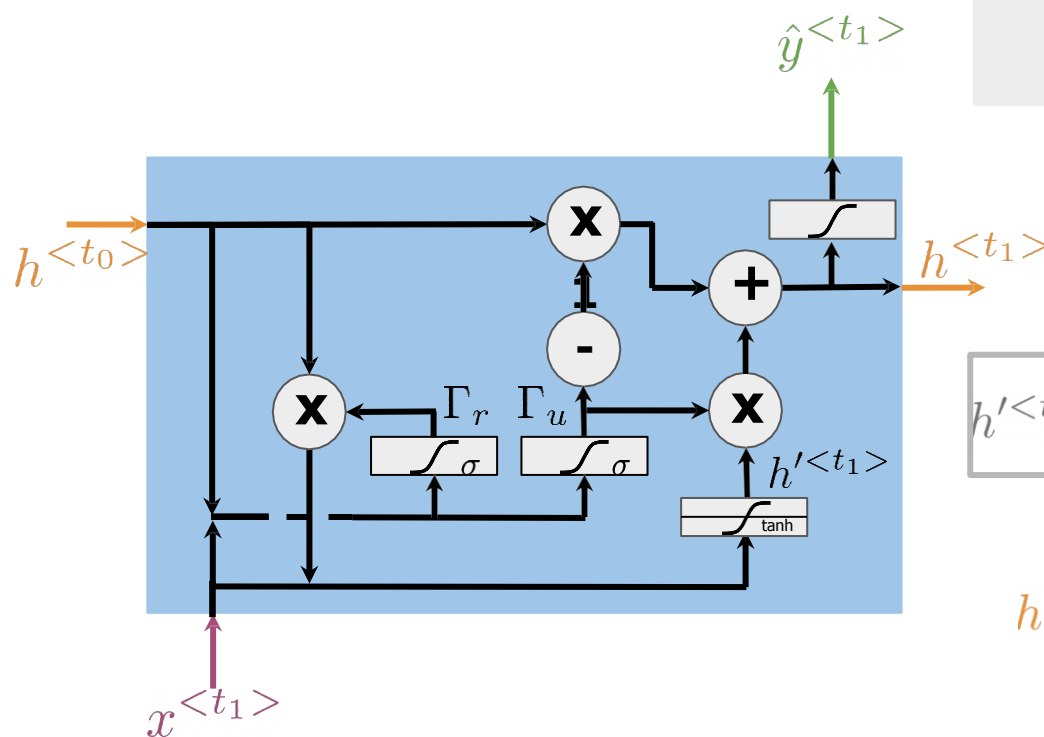
Gated Recurrent Units

"Ants are really interesting. They are everywhere."



Relevance and update gates to remember important prior information

Gated Recurrent Unit



Gates to keep/update relevant information in the hidden state

$$\begin{aligned}\Gamma_r &= \sigma(W_r[h^{<t_0>}, x^{<t_1>}] + b_r) \\ \Gamma_u &= \sigma(W_u[h^{<t_0>}, x^{<t_1>}] + b_u)\end{aligned}$$

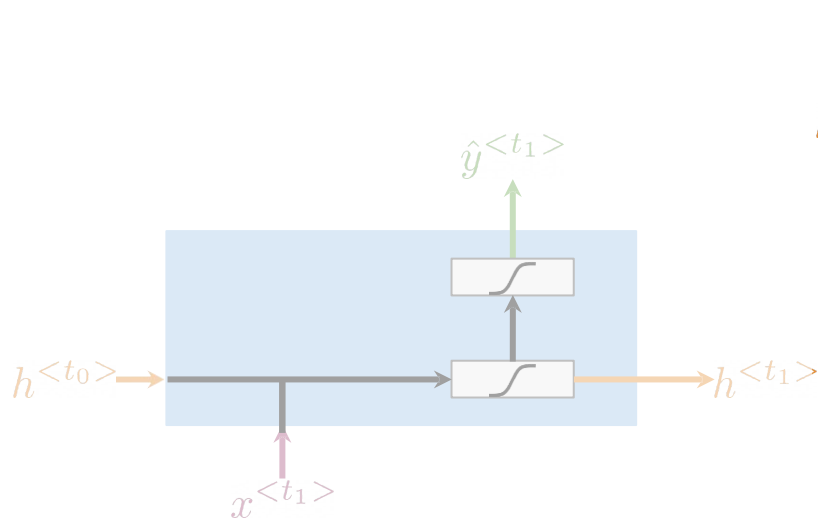
$$h'^{<t_1>} = \tanh(W_h[\Gamma_r * h^{<t_0>}, x^{<t_1>}] + b_h)$$

Hidden state candidate

$$h^{<t_1>} = (1 - \Gamma_u) * h^{<t_0>} + \Gamma_u * h'^{<t_1>}$$

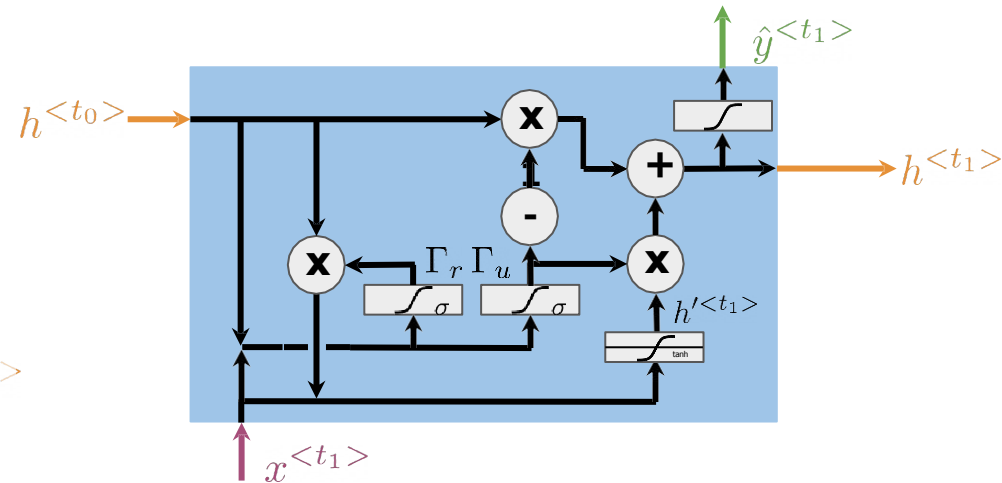
$$\hat{y}^{<t_1>} = g(W_y h^{<t_1>} + b_y)$$

Vanilla RNN vs GRUs



$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$

$$\hat{y}^{<t>} = g(W_{y_h}h^{<t>} + b_y)$$



$$\Gamma_u = \sigma(W_u[h^{<t_0>}, x^{<t_1>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[h^{<t_0>}, x^{<t_1>}] + b_r)$$

$$h'^{<t_1>} = \tanh(W_h[\Gamma_r * h^{<t_0>}, x^{<t_1>}] + b_h)$$

$$h^{<t_1>} = (1 - \Gamma_u) * h^{<t_0>} + \Gamma_u * h'^{<t_1>}$$

$$\hat{y}^{<t_1>} = g(W_y h^{<t_1>} + b_y)$$

Feature	LSTM	GRU
Gates	3 (i, f, o)	2 (z, r)
Memory Cell	Yes	No
Complexity	Higher	Lower
Training Speed	Slower	Faster
Performance	Great for long sequences	Similar or better on short tasks

Tip: Try GRU first for faster results; switch to LSTM if performance suffers.

NLP Tasks

- ▶ Language Modeling
- ▶ Machine Translation
- ▶ Sentiment Analysis
- ▶ Chatbots

Audio & Time Series

- ▶ Music Generation
- ▶ Speech Recognition
- ▶ Anomaly Detection

Video & Sequential Vision

- ▶ Action Recognition
- ▶ Video Captioning

Even with GRU/LSTM:

- ▶ Still sequential → Hard to parallelize
- ▶ Struggle with very long-range dependencies
- ▶ Architectural complexity
- ▶ Hard to interpret gate decisions
- ▶ Require lots of training data

- ▶ **Transformers:** Fully parallelized sequence modeling using attention
- ▶ **Efficient Attention:** Longformer, Linformer, etc. for long sequences
- ▶ **Neural Memory Networks:** Explicit memory read/write
- ▶ **Recurrent Attention Models**
- ▶ **Hybrid Architectures:** RNN + CNN + Attention

RNNs are still used in edge devices for efficient modeling

- ▶ RNNs struggle with long dependencies due to vanishing/exploding gradients
- ▶ GRU and LSTM improve memory retention using gating mechanisms
- ▶ GRU is simpler and faster; LSTM is more expressive
- ▶ Attention and transformers now dominate, but RNNs remain relevant in many domains

These slides have been adapted from

- Younes Mourri & Lukasz Kaiser, [Natural Language Processing Specialization, DeepLearning.AI](#)

Foundational Papers:

- ▶ Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
- ▶ Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder–Decoder with GRU*. EMNLP.
- ▶ Pascanu, R., Mikolov, T., & Bengio, Y. (2013). *On the difficulty of training RNNs*. ICML.
- ▶ Bengio, Y., Simard, P., & Frasconi, P. (1994). *Learning long-term dependencies*. IEEE Transactions on Neural Networks.
- ▶ Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS.

Resources:

- ▶ Karpathy's RNN Blog:
<https://karpathy.github.io/2015/05/21/rnn-effectiveness/>
- ▶ CS231n Lecture Notes on RNNs and LSTM
- ▶ DeepLearning.ai NLP Specialization – Coursera
- ▶ MIT 6.S191 Deep Learning Lecture Slides

Credits

Dr. Prashant Aparajeya

Computer Vision Scientist — Director(AISimply Ltd)

p.aparajeya@aisimply.uk

This project benefited from external collaboration, and we acknowledge their contribution with gratitude.